
An Exploration of Non-Euclidean Gradient Descent: Muon and its Many Variants

Michael Crawshaw^{1,2} Chirag Modi³ Mingrui Liu¹ Robert M. Gower²

Abstract

To define a steepest descent method over a neural network, we need to choose a norm for each layer, a way to aggregate these norms across layers, and whether to use normalization. We systematically explore different alternatives for aggregating norms across layers, both formalizing existing combinations of Adam and the recently proposed Muon as a type of non-Euclidean gradient descent, and deriving new variants of the Muon optimizer. Through a comprehensive experimental evaluation of the optimizers within our framework, we find that Muon is sensitive to the choice of learning rate, whereas a new variant we call MuonMax is significantly more robust. We then show how to combine any non-Euclidean gradient method with model based momentum (known as Momo). The new Momo variants of Muon are significantly more robust to hyperparameter tuning, and often achieve a better validation score. Thus for new tasks, where the optimal hyperparameters are not known, we advocate for using Momo in combination with MuonMax to save on costly hyperparameter tuning.

1. Introduction

The recently proposed Muon optimizer (Jordan et al., 2024b) has generated increasing interest due to its efficiency for training language models (Pethick et al., 2025; Liu et al., 2025). This algorithm was initially introduced (Bernstein & Newhouse, 2024a; Jordan et al., 2024b) and often interpreted (Pethick et al., 2025; Kovalev, 2025; Fan et al., 2025) as steepest descent with respect to the spectral norm for each weight matrix in a neural network.

¹Department of Computer Science, George Mason University

²Center for Computational Mathematics, Flatiron Institute ³Center for Cosmology and Particle Physics, New York University. Correspondence to: Michael Crawshaw <mcrawsha@gmu.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

However, this interpretation does not entirely apply for practical implementations of Muon. In practice, Muon is used side-by-side with another optimizer, where hidden weight matrices are trained with Muon, and all other parameters by Adam (Jordan et al., 2024b; Liu et al., 2025) or SignSGD (Pethick et al., 2025). We will refer to this combination as MuonAdam throughout, see Algorithm 1 in Appendix A. Furthermore, for the weight matrices only the normalized version of Muon has been explored in practice.

Here we aim to strengthen the theoretical foundation of MuonAdam and develop new optimizers by systematically investigating different design choices which have not been explored. We introduce a framework for steepest descent on the entire space of network parameters, which involves a norm for each layer, an outer norm to aggregate layer-wise norms into a *product norm* on the entire parameter space, and whether to normalize updates. This framework encompasses MuonAdam and other variations, which provides a principled interpretation of these algorithms as genuine steepest descent on the entire space of network parameters, and also opens a design space for previously unexplored Muon-type algorithms.

One unexplored variant is what we call MuonMax, that arises from a new outer norm and does not use update normalization. MuonMax’s updates depend on the nuclear norm of every weight matrix’s momentum, which is slightly less efficient per-step than Muon. To make MuonMax more efficient, we introduce a stale approximation of these nuclear norms, which requires near-identical memory and 5% additional wall-clock time per step as Muon. This approximation has negligible effect on the loss and can be used for many other methods.

Now that we can frame MuonAdam and other variants as a type of steepest descent, we can import other tools used for steepest descent gradient methods. One such tool is Momo (Schaipp et al., 2024), an adaptive step size based on model truncation (Asi & Duchi, 2019b) that increases robustness to learning rate tuning. We extend the Momo step size to general steepest descent for arbitrary norms and subsequently apply it to the algorithms in our framework.

We perform a systematic evaluation of many algorithms in our framework for training GPT models with up to

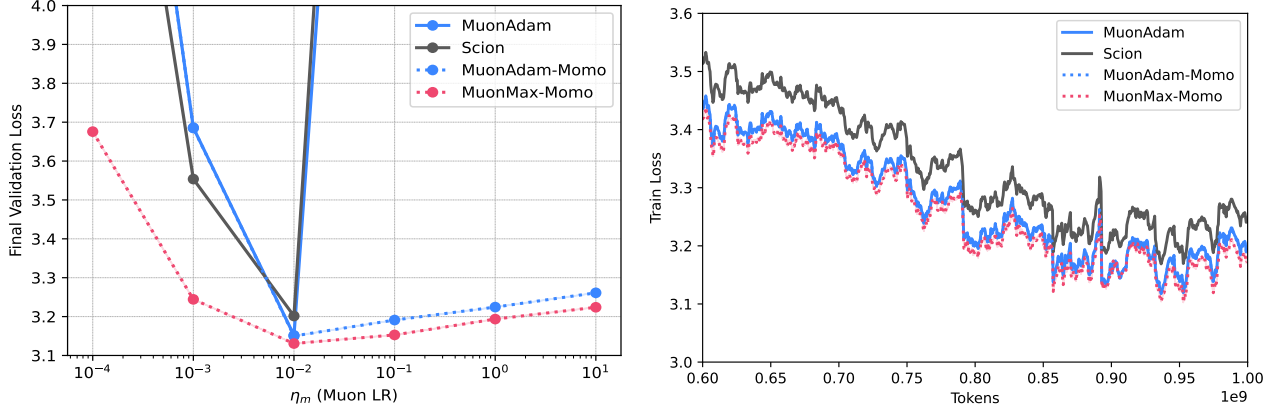


Figure 1. Learning rate sweep for training GPT2-Large (774M params) on SlimPajama with 1B tokens. **Left:** Final validation loss for various learning rates. MuonAdam and Scion require precise tuning, whereas our MuonAdam-Momo and MuonMax-Momo achieve low loss for a significantly wider range of learning rates. **Right:** Training loss (with tuned LR) for the last 40% of steps.

774M parameters for language modeling on the FineWeb (Penedo et al., 2024) and SlimPajama (Soboleva et al., 2023) datasets with up to 6B tokens. We find that MuonMax-Momo consistently matches or outperforms MuonAdam and Scion (Pethick et al., 2025) while enjoying a much larger range of competitive learning rates, meaning that MuonMax-Momo is much less sensitive to tuning. We also observe that Momo increases tuning robustness for all variations and that our stale nuclear norm approximation causes negligible change in performance, while decreasing wall-clock time per iteration. Similar robustness is observed in image classification experiments. Our contributions are:

1. **Formalizing MuonAdam.** We introduce a steepest descent framework that encompasses the practical implementation of Muon (with Adam used for a subset of parameters), demonstrating that even these side-by-side optimizers can be interpreted as steepest descent with respect to certain norms on the space of *all* network parameters. This solidifies the theoretical foundation for practical variants of Muon, and sheds light on unexplored aspects of Muon’s design. Our framework also includes Scion and other existing variants.
2. **Defining non-Euclidean Momo.** We show how to incorporate the adaptive step size Momo with any steepest descent algorithm, which we find significantly increases robustness to the learning rate tuning.
3. **MuonMax: New practical, robust variant of Muon.** We propose a new optimizer, MuonMax, which arises as steepest descent in a novel product norm. With a stale approximation of the nuclear norm of each layer’s momentum, MuonMax has near-identical memory cost and 5% additional wall-clock time per iteration compared to Muon.

4. **Systematic Evaluation.** We evaluate optimizers in our framework for language modeling (GPT2-Small, GPT2-Large, and Qwen2-MoE with FineWeb and SlimPajama data), and image classification (ResNet20 with CIFAR10, and ResNet110 with CIFAR100). Across settings, MuonMax-Momo consistently achieves a similar or lower loss than MuonAdam and baselines while widening the range of viable step sizes by several orders of magnitude.

Notation. We use $\langle \cdot, \cdot \rangle$ to denote the Euclidean inner product on $\mathbb{R}^d$ or on $\mathbb{R}^{d_1} \times \dots \times \mathbb{R}^{d_n}$ by naturally viewing elements of $\mathbb{R}^{d_1} \times \dots \times \mathbb{R}^{d_n}$ as elements of $\mathbb{R}^{d_1 + \dots + d_n}$. Note that for matrices, which can also be viewed as elements of $\mathbb{R}^{mn}$, this inner product is consistent with the trace inner product.

2. Related Work

Muon. The use of *Spectral descent*, that is steepest descent with respect to the spectral norm, on deep neural networks dates back to Carlson et al. (2015a;b). Muon is the combination of spectral descent with momentum (Bernstein & Newhouse, 2024a), and a carefully crafted polynomial algorithm for computing the polar factor (Jordan et al., 2024b). Recent work has designed an optimal such polynomial algorithm for the polar factor called PolarExpress (Amsel et al., 2025), which we use in our Muon implementation. Pethick et al. (2025) introduced Scion, which uses SignSGD with momentum (instead of Adam) to train non-matrix parameters. Liu et al. (2025) scaled Muon to train a 16B parameter language model with 5.7T tokens. Several works have developed theory of Muon’s convergence (Li & Hong, 2025; Kovalev, 2025; Riabinin et al., 2025) and implicit bias (Tsilivis et al., 2025; Fan et al., 2025).

Other lines of work that focus on explaining why spectral methods could converge faster include (Davis & Drusvyatskiy, 2025) which analyses one step descent under a relative smoothness condition, (Su, 2025) that shows that when the curvature grows sufficiently fast, that spectral dampening (with Muon being the extreme case) is an optimal method under an isotropic model. Spectral descent has also been motivated through its equivariance under the symmetry group acting on the corresponding weight block (Lau & Su, 2026).

Most similar to ours is the line of work on the modular norm (Bernstein & Newhouse, 2024a; Large et al., 2024; Bernstein & Newhouse, 2024b), which also argues that we should perform steepest descent on the entire space of parameters, not separately at each layer. These works focus on the modular norm, which enables Lipschitz continuity of the neural network in both weights and inputs. In this work, we take an orthogonal approach, where we theoretically consider steepest descent on product spaces with general norms, and numerically investigate many choices of norms on these spaces. We are not aware of any existing evaluation of steepest descent with respect to the modular norm.

Model Truncation. Gradient descent can be viewed as using the local linearization of the loss as a *model* of the loss itself. If we know a lower bound of the loss, for instance most loss functions are positive, then we can improve this linear model by *truncating* the model at this lower bound (Asi & Duchi, 2019a). Follow-up work emphasizes the importance of such model choices in stochastic optimization (Asi & Duchi, 2019b), and extends the framework to minibatch settings (Asi et al., 2020). Using model truncation often leads to methods that are more stable and easier to tune (Loizou et al., 2021; Davis & Drusvyatskiy, 2019; Meng & Gower, 2023; Schaipp et al., 2023). Recently Schaipp et al. (2024) showed how to combine momentum with model truncation. Furthermore, Chen et al. (2022) combine parameter-free coin betting methods with truncated models.

3. Steepest Descent on Neural Networks

Let $F : \mathbb{R}^d \rightarrow \mathbb{R}$ be the loss function, and $\|\cdot\|$ be any norm on $\mathbb{R}^d$. We define the *Linear Minimization Oracle* (LMO) and the *dual norm* as

$$\text{LMO}_{\|\cdot\|}(v) = \arg \min_{\|u\|=1} \langle u, v \rangle, \quad \|v\|_* = \max_{\|u\|=1} \langle u, v \rangle, \quad (1)$$

respectively. When the norm is clear from context, we omit the subscript and write LMO. Throughout we denote the stochastic gradient at step t by g_t , and the momentum buffer by m_t , which is an exponential moving average of stochastic gradients, i.e. $m_t = \beta m_{t-1} + (1 - \beta)g_t$ for $\beta \in [0, 1)$.

3.1. Constrained vs Regularized Steepest Descent

For a single weight matrix, the Muon update is often motivated as the LMO (Pethick et al., 2025) of the momentum buffer m_t at each step, with respect to the spectral norm. The following proposition shows that for a general norm, updating in the direction of $\text{LMO}(m_t)$ is equivalent to minimizing a first-order Taylor approximation of F around w_t , with a constraint on the update’s norm and approximating $\nabla F(w_t) \approx m_t$.

Proposition 3.1. [Constrained Steepest Descent] The CSD update is given by

$$\begin{aligned} w_{t+1} &= \arg \min_{\|w - w_t\| \leq \eta} \{F(w_t) + \langle m_t, w - w_t \rangle\} \\ &= w_t + \eta \text{LMO}(m_t). \end{aligned} \quad (2)$$

The ball constraint above ensures that we only use the Taylor approximation close to its center w_t . Another natural choice is to use regularization instead of a constraint as follows.

Proposition 3.2. [Regularized Steepest Descent] The RSD update is given by

$$\begin{aligned} w_{t+1} &= \arg \min_w \left\{ F(w_t) + \langle m_t, w - w_t \rangle \right. \\ &\quad \left. + \frac{1}{2\eta} \|w - w_t\|^2 \right\} \\ &= w_t + \eta \|m_t\|_* \text{LMO}(m_t) \end{aligned} \quad (3)$$

In the case without momentum (i.e. $\beta = 0$), both of these algorithms have appeared throughout the literature under the name steepest descent, but the recent line of work around Muon (Jordan et al., 2024b; Bernstein & Newhouse, 2024b; Pethick et al., 2025; Liu et al., 2025) has mostly focused on the constrained variant. To the best of our knowledge, the only work which considered the regularized variant over the space of all parameters was Bernstein & Newhouse (2024a). Lau et al. (2025) use the regularized interpretation of Muon on a per layer basis instead of the entire product space.

Notice that CSD and RSD have the same update direction, but with regularization the update magnitude is scaled by the dual norm of the momentum. Therefore, the primal norm of the update $\|w_{t+1} - w_t\|$ is η for CSD and $\eta \|m_t\|_*$ for RSD. In that way, CSD enforces a *normalized update*.

3.2. Product Norms

Muon’s update applies constrained steepest descent *separately* to each layer of a neural network. Instead, we argue that steepest descent should operate directly on the space of all parameters of a neural network, and to do so we must define a norm on this space. Instead of flattening

parameters into a single vector, we consider the product $\mathbf{W} = (\mathbf{w}^1, \mathbf{w}^2, \dots, \mathbf{w}^n)$ of network parameters (where each $\mathbf{w}^i$ could be a flattened weight matrix, a bias vector, etc). We choose a norm $\|\cdot\|_{(i)}$ for parameter $\mathbf{w}^i$, and an *outer norm* to aggregate the layer-wise norms into a single norm on the product space. For example, we could choose our outer norm as $\|\cdot\|_\infty$ or $\|\cdot\|_2$, leading to the following product norms, respectively:

$$\|\mathbf{W}\|_\infty := \max_{1 \leq i \leq n} \|\mathbf{w}^i\|_{(i)}, \quad \|\mathbf{W}\|_2 := \sqrt{\sum_{i=1}^n \|\mathbf{w}^i\|_{(i)}^2}.$$

Computing a steepest descent update with respect to a product norm requires: the linear minimization oracle (LMO) and the dual norm of the product norm. As we show next, both can be expressed in terms of the underlying per-parameter norms and the outer norm used to aggregate them.

Lemma 3.3. [LMO and Dual of Product Norms] For each $i \in [n]$, let g_i be a norm on $\mathbb{R}^{d_i}$, let f be a norm on $\mathbb{R}^n$, and denote their dual norms as g_i^* and f^* . Then the product norm $h : \mathbb{R}^{d_1} \times \dots \times \mathbb{R}^{d_n} \rightarrow \mathbb{R}$ defined by

$$h(\mathbf{w}^1, \dots, \mathbf{w}^n) = f(g_1(\mathbf{w}^1), \dots, g_n(\mathbf{w}^n))$$

is indeed a norm, and its LMO and dual are given by

$$\begin{aligned} \text{LMO}_h(\mathbf{w}^1, \dots, \mathbf{w}^n) &= (\phi_1 \text{LMO}_{g_1}(\mathbf{w}^1), \dots, \phi_n \text{LMO}_{g_n}(\mathbf{w}^n)) \\ h_*(\mathbf{w}^1, \dots, \mathbf{w}^n) &= f_*(g_1^*(\mathbf{w}^1), \dots, g_n^*(\mathbf{w}^n)), \end{aligned}$$

where $(\phi_1, \dots, \phi_n) := -\text{LMO}_{f^*}(g_1^*(\mathbf{w}^1), \dots, g_n^*(\mathbf{w}^n))$.

With this lemma, we can quickly compute steepest descent updates (both constrained and regularized) with respect to the product norms $\|\cdot\|_\infty$, $\|\cdot\|_2$, or any other product norm, by plugging the LMO and dual of each product norm into the steepest descent definitions (Equation 2 and Equation 3).

Denoting by $\mathbf{m}_t^i$ the momentum buffer of parameter i , the resulting update for each parameter $\mathbf{w}^i$ can be written as

$$\mathbf{w}_{t+1}^i = \mathbf{w}_t^i + \eta \lambda_t^i \text{LMO}_{\|\cdot\|_{(i)}}(\mathbf{m}_t^i),$$

where λ_t^i depends on CSD vs. RSD and the outer norm:

$$(\text{CSD}, \|\cdot\|_\infty): \quad \lambda_t^i = 1 \quad (4)$$

$$(\text{RSD}, \|\cdot\|_\infty): \quad \lambda_t^i = \sum_{j=1}^n \|\mathbf{m}_t^j\|_{(j),*} \quad (5)$$

$$(\text{CSD}, \|\cdot\|_2): \quad \lambda_t^i = \frac{\|\mathbf{m}_t^i\|_{(i),*}}{\sqrt{\sum_{j=1}^n \|\mathbf{m}_t^j\|_{(j),*}^2}} \quad (6)$$

$$(\text{RSD}, \|\cdot\|_2): \quad \lambda_t^i = \|\mathbf{m}_t^i\|_{(i),*}. \quad (7)$$

Note that the update direction for each parameter $\mathbf{w}_t^i$ is always the LMO of $\mathbf{m}_t^i$, regardless of the choice of outer

norm. However, the magnitude of each parameter's update is determined by the outer norm and parameter-wise dual norms $\|\mathbf{m}_t^j\|_{(j),*}$. Therefore, different choices of the outer norm amount to different *parameter-wise learning rates*.

3.3. Incorporating Adam

Now we show how to represent the hybrid MuonAdam as a steepest descent method. For parameters θ , the Adam update, where all vector operations are element-wise¹, is

$$\theta_{t+1} = \theta_t - \eta \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t + \epsilon}}, \quad \mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t^2.$$

Adam can be interpreted as steepest descent in two ways.

Proposition 3.4. The t -th update of Adam is the CSD with step size η with respect to the norm:

$$\|\theta\|_{\text{ada}\infty} := \|\text{Diag}\left(\frac{\sqrt{\mathbf{v}_t + \epsilon}}{|\mathbf{m}_t|}\right)\theta\|_\infty$$

Proposition 3.5. The t -th update of Adam is the RSD with step size η with respect to the norm:

$$\|\theta\|_{\text{ada}2} := \sqrt{\langle \text{Diag}(\sqrt{\mathbf{v}_t + \epsilon})\theta, \theta \rangle}$$

Thus Adam can be interpreted as an adaptive trust-region sign descent (Balles & Hennig, 2018; Orvieto & Gower, 2025) or preconditioned gradient descent (Schaipp et al., 2024). A distinctive feature of these forms of steepest descent is that the norm changes over iterations. Such algorithms do not fit into the standard theoretical analysis of steepest descent (Kelner et al., 2014), but this will not affect our goal of formulating MuonAdam as steepest descent.

3.4. The Whole Framework

For a given neural network, we partition the parameters as $\mathbf{W} = (\mathbf{W}^1, \dots, \mathbf{W}^L, \theta)$, where $\mathbf{W}^1, \dots, \mathbf{W}^L$ are the hidden weight matrices and θ contains the remaining parameters flattened into a single vector. MuonAdam applies LMO updates w.r.t. the spectral norm for the hidden weight matrices, and uses Adam for the remainder of the parameters, with two separate learning rates for these side-by-side optimizers, shown in Algorithm 1 (Appendix A).

Proposition 3.6. MuonAdam (Algorithm 1) is exactly CSD with step size η_m with respect to

$$\|\mathbf{W}\|_{\text{muon}} = \max \left(\max_{\ell \in [L]} \|\mathbf{W}^\ell\|_{2 \rightarrow 2}, \frac{\eta_m}{\eta_b} \|\theta\|_{\text{ada}\infty} \right)$$

The coefficient η_m/η_b effectively allows for the use of dif-

¹We omit bias correction, since the bias can be removed by correctly initializing momentum (Schaipp et al., 2024). In any case it has little effect on performance (Orvieto & Gower, 2025).

Extra Memory	Extra Time	Stale Norms
<pre> d = 0 lmos = {} for i in range(1, L+1): lmos[i] = -polar(M[i]) d -= lmos[i].dot(M[i]) for i in range(1, L+1): W[i] += eta * d * lmos[i]</pre>	<pre> d = 0 for i in range(1, L+1): lmo = -polar(M[i]) d -= lmo.dot(M[i]) for i in range(1, L+1): lmo = polar(M[i]) W[i] += eta * d * lmo</pre>	<pre> new_d = 0 for i in range(1, L+1): lmo = -polar(M[i]) new_d -= lmo.dot(M[i]) W[i] += eta * old_d * lmo old_d = new_d</pre>

Figure 2. Sketch of different implementations for an update with learning rates that are non-separable across layers.

ferent learning rates for hidden weight matrices compared to all other parameters; this is a crucial feature of Muon’s speedrun implementation (Jordan et al., 2024b) and of other variations (Pethick et al., 2025; Liu et al., 2025).

Proposition 3.6 shows the precise sense in which MuonAdam is a steepest descent algorithm: it is constrained steepest descent with respect to a particular product norm that aggregates the spectral norm of each hidden weight matrix and an adaptive ℓ_∞ norm for all other parameters. This still leaves several other valid choices within our general steepest descent framework to explore: whether to use constrained or regularized steepest descent, which outer norm to use (e.g. $\|\cdot\|_\infty, \|\cdot\|_2$), and which norm to assign to the non-matrix parameters (e.g. $\|\cdot\|_{\text{ada}\infty}, \|\cdot\|_{\text{ada}2}, \|\cdot\|_\infty$).

These three factors yield a design space for Muon-type algorithms, all of which are founded on the principle of steepest descent, and most of which are unexplored. Among these algorithms are several existing variations of Muon, including Scion (Pethick et al., 2025) and PolarGrad (Lau et al., 2025) (see Appendix A.1 for more detail).

Stale dual norms. Many of the updates we have presented so far require calculating dual norms of the momentum buffers (e.g. Equation 5 through Equation 7). If that norm is the spectral norm, this amounts to computing the nuclear norm of the momentum, which may appear costly, but actually the dual norm is easy to compute once we have computed the LMO, since $\|v\|_* = \langle -\text{LMO}(v), v \rangle$.

However, in the case that updates are not separable across parameters, computing the dual norms of each momentum buffer in this way requires either additional memory (to store the layer-wise LMOs) or additional time (to compute the LMOs twice). To see why, consider the example of RSD with the $\|\cdot\|_\infty$ outer norm (Equation 5), and assume for simplicity that all parameters are assigned the spectral norm. For each layer i , the update

$$W_{t+1}^i = W_t^i - \eta \left(\sum_{j=1}^L \|M_t^j\|_{\text{nuc}} \right) \text{polar}(M_t^i) \quad (8)$$

cannot be executed until $\|M_t^j\|_{\text{nuc}} = \langle \text{polar}(M_t^j), M_t^j \rangle$

has been computed for every layer j . Crucially, the polar factors are used twice here: once to compute dual norms, and again to update weights. So, we can either store the polar factors for reuse (extra memory), or compute them twice (extra time); these options are sketched in the first two columns of Figure 2.

The first option requires additional memory proportional to the size of the network, while the second option doubles the wall-clock time needed to compute polar factors. As an efficient approximation, we propose to reuse momentum dual norms from the previous step (third column of Figure 2), which can be implemented without storing or recomputing polar factors, and only requires a single additional scalar of memory for each layer. We found in our experiments that using these “stale” dual norms had near negligible effect on performance. Informally, we expect this approximation to work on the grounds that the momentum doesn’t change too drastically in a single step, since

$$\begin{aligned} m_t - m_{t-1} &= \beta m_{t-1} + (1 - \beta) g_t - m_{t-1} \\ &= (1 - \beta)(g_t - m_{t-1}) \end{aligned}$$

has small magnitude when β is close to 1.

A New Product Norm. MuonMax, one of our proposed algorithms, is regularized steepest descent with respect to the following norm:

$$\|W\|_{\text{MM}} = \sqrt{\left(\max_{\ell \in [L]} \|W^\ell\|_{2 \rightarrow 2} \right)^2 + \frac{\eta_m}{\eta_b} \|\theta\|_{\text{ada}2}^2} \quad (9)$$

This norm comes from assigning $\|\cdot\|_{\text{ada}2}$ to the non-matrix parameters, maximum spectral norm over layers for the matrix parameters, then aggregating both using the standard Euclidean norm. We chose this norm as a way to run Adam on non-matrix parameters while running a non-separable version of Muon (as in Equation 8) for the hidden weight matrices. We denote the corresponding outer norm as

$$\|(v_1, \dots, v_L, v_{L+1})\|_{\text{hyb}} = \sqrt{\left(\max_{\ell \in [L]} v_\ell \right)^2 + \frac{\eta_m}{\eta_b} v_{L+1}^2}.$$

The final version of MuonMax (including model truncation) will be given in the following section.

4. Model Truncation

The solid theoretical footing offered by our framework also enables practical benefits: techniques designed for SGD (or normalized SGD) can now be easily adapted for Muon-type algorithms by generalizing to arbitrary norms. In this section, we generalize Momo (Schaipp et al., 2024) for steepest descent with respect to arbitrary norms, which can then be immediately applied to any algorithm in our framework.

Recall that steepest descent is motivated by locally minimizing a first-order Taylor approximation of the loss around the current iterate. Momo makes use of *model truncation* (Asi & Duchi, 2019b), which leverages knowledge of a loss lower bound F_* to construct an approximation of the loss which is more accurate than a Taylor approximation while still having a simple closed form for the minimizer. In Momo, this model also incorporates information from the history of gradients and losses through momentum.

Denote $\rho_{i,t} = (1 - \beta)\beta^{t-i}$, so that $\mathbf{m}_t = \sum_{i=0}^t \rho_{i,t} \mathbf{g}_i$, and denote by $F_t(\mathbf{w}_t)$ the minibatch loss at step t . Then for each t , we can build a model of the loss around $\mathbf{w}_t$ as a weighted average of first-order Taylor approximations centered at previous iterates. Denoting

$$\tilde{F}_t := \sum_{i=0}^t \rho_{i,t} (F_i(\mathbf{w}_i) + \langle \mathbf{g}_i, \mathbf{w}_t - \mathbf{w}_i \rangle),$$

this approximation can be written

$$\begin{aligned} F(\mathbf{w}) &\approx \sum_{i=0}^t \rho_{i,t} (F_i(\mathbf{w}_i) + \langle \mathbf{g}_i, \mathbf{w} - \mathbf{w}_i \rangle) \\ &= \sum_{i=0}^t \rho_{i,t} (F_i(\mathbf{w}_i) + \langle \mathbf{g}_i, \mathbf{w}_t - \mathbf{w}_i \rangle) \\ &\quad + \sum_{i=0}^t \rho_{i,t} \langle \mathbf{g}_i, \mathbf{w} - \mathbf{w}_t \rangle \\ &= \tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle. \end{aligned}$$

Since $F(\mathbf{w}) \geq F_*$ for all $\mathbf{w}$, we can improve our model by truncating, or clipping, it at F_* :

$$F(\mathbf{w}) \approx \max \left(\tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle, F_* \right).$$

Our truncated steepest descent methods, shown below, arise from minimizing this truncated model either with a norm constraint or with squared norm regularization.

Proposition 4.1. [Constrained Momo] The ball constrained truncated model update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\|\mathbf{w} - \mathbf{w}_t\| \leq \eta} \left\{ \max \left(\tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle, F_* \right) \right\} \\ &= \mathbf{w}_t + \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right) \text{LMO}(\mathbf{m}_t) \end{aligned}$$

The arg min above can have multiple solutions: we take the one that has minimal distance to $\mathbf{w}_t$.

Proposition 4.2. [Regularized Momo] The regularized truncated model update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\mathbf{w}} \left\{ \max \left(\tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle, F_* \right) \right. \\ &\quad \left. + \frac{1}{2\eta} \|\mathbf{w} - \mathbf{w}_t\|^2 \right\} \\ &= \mathbf{w}_t + \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*^2} \right) \|\mathbf{m}_t\|_* \text{LMO}(\mathbf{m}_t) \end{aligned}$$

$\tilde{F}_t$ depends on the history of gradients and losses, but it can be computed with a single scalar running average, which incurs negligible additional costs compared to non-truncated steepest descent. Pseudocode for both Momo variations is shown in Algorithm 2.

Now that we have shown how to use Momo with respect to any norm, we can immediately combine Momo with any steepest descent algorithm in our framework, including MuonAdam. For example, our proposed algorithm MuonMax-Momo (Algorithm 3 in Appendix B) can be written as Regularized Momo w.r.t. $\|\cdot\|_{\text{MM}}$ (defined in Equation 9) with stale dual norm approximations.

Proposition 4.3. [MuonMax-Momo] Regularized Momo with respect to the norm $\|\cdot\|_{\text{MM}}$ as defined in equation 9 has the following closed form:

$$\begin{aligned} d_t &= \sqrt{\left(\sum_{\ell=1}^L \|\mathbf{M}_t^\ell\|_{\text{nuc}} \right)^2 + \frac{\eta_b}{\eta_m} \left\| \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}} \right\|_2^2} \\ \eta'_m &= \min \left\{ \eta_m, \frac{\tilde{F}_t - F_*}{d_t^2} \right\}, \quad \eta'_b = \min \left\{ \eta_b, \frac{\eta_b}{\eta_m} \frac{\tilde{F}_t - F_*}{d_t^2} \right\} \\ \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta'_m \left(\sum_{\ell=1}^L \|\mathbf{M}_t^\ell\|_{\text{nuc}} \right) \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta'_b \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}}. \end{aligned}$$

The update in Proposition 4.3 matches that of Algorithm 3 except for the use of stale dual norms in Algorithm 3.

5. Experiments

Here we provide a comprehensive evaluation of optimizers arising from our steepest descent framework for training language models. We include experiments for image classification in Appendix E. Each training run was executed on a single node with four NVIDIA H100 GPUs with 80GB memory each.

We start by tuning and evaluating 36 optimizer variations arising from different choices of normalization, outer norm, norm for the non-matrix parameters, and whether to use model truncation. For this initial phase of evaluating all variations, we use 1B tokens from the FineWeb dataset to train a GPT2-Small model with 124M params (Section 5.1).

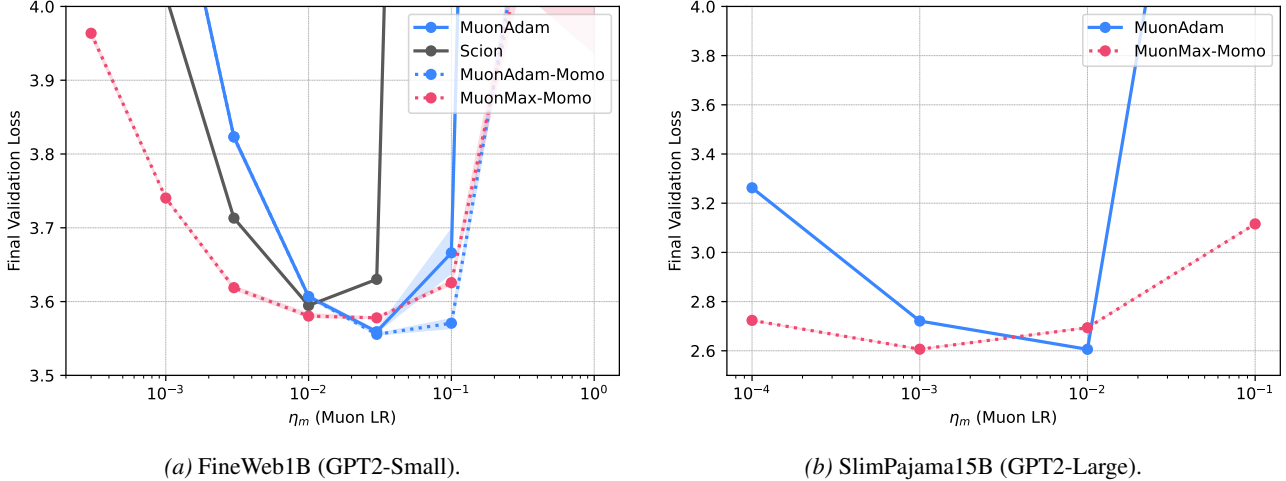


Figure 3. Final validation loss with varying learning rates on FineWeb1B (left) and SlimPajama15B (right). Our MuonAdam-Momo and MuonMax-Momo have wider basins than MuonAdam and Scion, indicating increased robustness to learning rate tuning.

We take the four best performing methods (MuonAdam, Scion, MuonAdam-Momo, MuonMax-Momo) and evaluate them for a GPT2-Large model with 774M params on the SlimPajama dataset (Section 5.2), first by thoroughly tuning all four algorithms with 1B tokens, then running a final evaluation of Muon and MuonMax-Momo with 15B tokens. Finally, in Section 5.3 we perform two studies: we examine the sensitivity of Momo variants to the choice of the loss lower bound F_* , and how stale nuclear norm approximations affect the final loss and wall-clock time.

We give further experiments in Appendix D, including additional baselines, an evaluation with 6B tokens, more metrics by which to compare baselines, experiments with Qwen2-MoE models, and an evaluation with varying batch size.

5.1. FineWeb Dataset

To identify the strongest methods within our framework, we thoroughly tune and evaluate 36 variations that arise from mixing and matching settings for the following design choices: constrained vs regularized steepest descent, outer norm ($\|\cdot\|_\infty, \|\cdot\|_2, \|\cdot\|_{\text{hyb}}$), norm for parameters besides hidden weight matrices ($\|\cdot\|_\infty, \|\cdot\|_{\text{ada}\infty}, \|\cdot\|_{\text{ada}2}$), and whether to apply model truncation.

Setup. For all variations, we run one epoch of training with 1B tokens from the FineWeb dataset (Penedo et al., 2024), using the GPT-2 Small architecture (124M params) from modded-nanogpt (Jordan et al., 2024a). Each algorithm in our framework has two learning rates: η_m for the hidden weight matrices (which we call the Muon learning rate) and η_b for everything else (which we call the base learning rate). Similarly as prior works, we found that optimization speed is sensitive to the ratio between these two

learning rates (Pethick et al., 2025; Liu et al., 2025). Due to the computational cost of performing a double grid search, we opt to tune with an iterated grid search; for each algorithm, we fix η_b while tuning η_m , then fix η_m at the tuned value while tuning η_b . See Appendix C for a complete specification of the tuning protocol and other implementation details. For all Momo variations, we set the lower bound $F_* = 3.2$, and conduct a sensitivity analysis of this hyperparameter in Section 5.3.

Results. The final loss for each variation is shown in Tables 2 and 3 of Appendix D. For the best performing variations (MuonAdam, Scion, MuonAdam-Momo, and MuonMax-Momo), we additionally evaluate the sensitivity to learning rate tuning by running each algorithm with LRs $(\rho\eta_m, \rho\eta_b)$, where (η_m, η_b) are the previously tuned LRs and ρ varies over $\{0.03, 0.1, 0.3, 1, 3, 10, 30, 100\}$, with three random seeds each (Figure 3a). Table 4 in Appendix D gives the mean and standard deviation of final validation loss for each algorithm with tuned LRs. For these runs, MuonAdam-Momo and MuonMax-Momo use stale nuclear norms.

In Figure 3a, we see that MuonAdam and MuonAdam-Momo achieve the smallest loss among all variations, though MuonAdam is much more sensitive to the learning rate. Both MuonAdam-Momo and MuonMax-Momo enjoy a much wider range of competitive learning rates compared with MuonAdam and Scion; for this search range, the proportion of LRs yielding loss less than 3.65 is 25% for MuonAdam and Scion, 50% for MuonMax-Momo, and 62.5% for MuonAdam-Momo. Also, Table 4 (Appendix D) shows that both of our Momo methods achieve a smaller variation in loss across random seeds compared with MuonAdam and Scion.

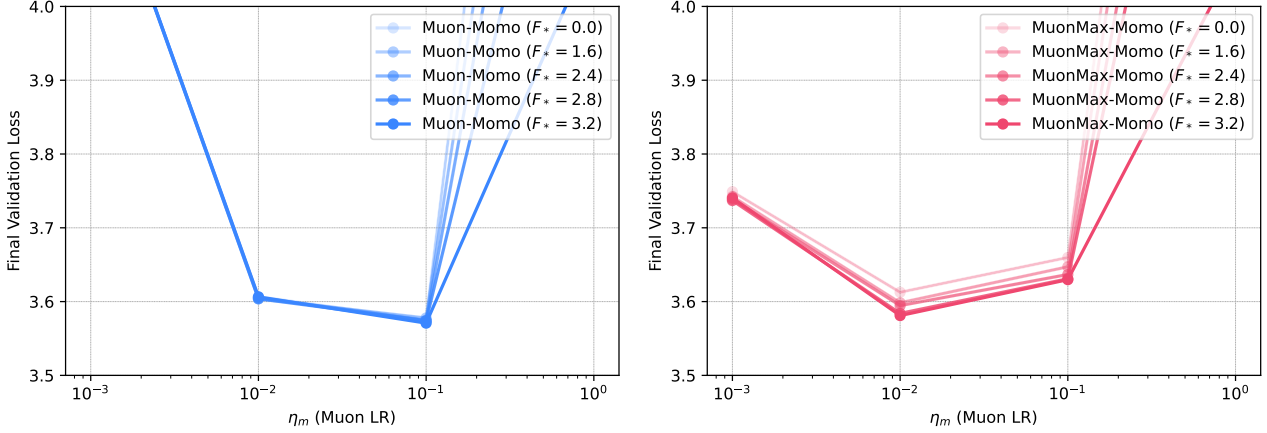


Figure 4. Sensitivity to loss lower bound F_* for model truncation (Fineweb1B).

We additionally include in Appendix D.1.1 an evaluation of four popular baselines which are not in the Muon family: AdamW (Loshchilov & Hutter, 2019), Lion (Chen et al., 2023), Adan (Xie et al., 2024), and Sophia (Liu et al., 2024). Overall, we find that all four of these algorithms lag behind the four best Muon variations, even after additional hyperparameter tuning, which is consistent with previous work (Jordan et al., 2024b; Pethick et al., 2025; Liu et al., 2025). See Appendix D.1.1 for a full discussion.

5.2. SlimPajama Dataset

Having identified MuonAdam, Scion, MuonAdam-Momo, and MuonMax-Momo as the strongest variations, we evaluate these methods for training the GPT2-Large architecture (774M params) using the SlimPajama dataset (Soboleva et al., 2023). We first evaluate all four algorithms for one epoch with 1B tokens, then evaluate MuonAdam and MuonMax-Momo for one epoch with 6B tokens, then 15B tokens.

Setup. Most aspects of training are the same as in Section 5.1, the main difference being the tuning protocol. To tune the two learning rates η_m and η_b , we run a double grid search for each algorithm, varying $\eta_m \in \{1e-4, 1e-3, 1e-2, 1e-1\}$ and $\eta_b \in \{1e-5, 1e-4, 1e-3, 1e-2\}$ for a total of 16 settings per algorithm. For the Momo variants, we set the lower bound $F_* = 2.8$ when training with 1B tokens and $F_* = 2.0$ when training with 6B and 15B tokens. We did not tune F_* , and based on the sensitivity analysis in Section 5.3, we expect that this hyperparameter does not have a large effect on final performance for tuned learning rates. Note that here both MuonAdam-Momo and MuonMax-Momo are using stale nuclear norm approximations.

Results. Figure 1 shows the final loss of each method with LR_s ($\rho\eta_m, \rho\eta_b$), where (η_m, η_b) are tuned LR_s and $\rho \in \{1e-$

2, 1e-1, 1, 1e1, 1e2, 1e3}. The sensitivity of each method with respect to both learning rates is shown for the full 2D grid in Figure 11 of Appendix D. We see in Figure 1 that MuonMax-Momo achieves the lowest loss of all methods, and that both Momo variations are extremely robust to the choice of learning rates. Both MuonAdam and Scion have quite narrow sensitivity curves, that is, shifting the optimal learning rates by a factor of 10 in either direction creates a large increase in final loss. In comparison, the final loss of MuonMax-Momo remains between 3.13 and 3.24 even as η_m varies over five orders of magnitude from 1e-3 to 10.

We see similar robustness of MuonMax-Momo when scaling up to 15B tokens. Due to the cost of re-tuning learning rates, we reuse the ratio η_m/η_b of the tuned learning rates from 1B training, and vary $\eta_m \in \{1e-4, 1e-3, 1e-2, 1e-1\}$ for MuonAdam and MuonMax-Momo. Figure 3b shows that with 15B tokens, MuonMax-Momo achieves a similar or lower loss than MuonAdam for every setting in this range, and generally exhibits less variation in the loss as the learning rates are shifted from their optimal values. Similar behavior is observed with 6B tokens, shown in Figure 5 of Appendix D.1.

5.3. Ablations

To probe the behavior of our proposed methods, we investigate: (1) how the choice of loss lower bound F_* affects the final validation loss of MuonAdam-Momo and MuonMax-Momo; (2) the effect of using stale nuclear norm approximations on the final validation loss and wall-clock time per iteration for several methods in our framework. Additionally, Appendix D.1.3 contains a evaluation of our proposed methods and baselines under varying batch size. In this section, we use the same setup as in Section 5.1 (GPT2-Small, FineWeb dataset, 1B tokens).

	MuonAdam	MuonMax	MuonAdam-Momo	Scion-Momo	MuonMax-Momo
Original	3.604 (1 $\times$)	3.791 (1.09 $\times$)	3.551 (1.10 $\times$)	3.597 (1.08 $\times$)	3.576 (1.11 $\times$)
Stale	-	3.768 (1.04 $\times$)	3.554 (1.04 $\times$)	3.594 (1.02 $\times$)	3.580 (1.05 $\times$)

Table 1. Effect of stale nuclear norm approximation on final loss and wall-clock time per-iteration compared to MuonAdam, which has no stale variant because it does not involve nuclear norms.

Sensitivity Analysis of F_* . Figure 4 shows the final loss of MuonAdam-Momo and MuonMax-Momo with various η_m , as the loss lower bound F_* varies over $\{0, 1.6, 2.4, 2.8, 3.2\}$. We see that the choice of F_* makes the biggest difference when η_m is larger than the optimal value. Interestingly, MuonAdam-Momo is quite robust to the choice of F_* , as the final loss is nearly identical for all values of F_* when $\eta_m \leq 0.1$. MuonMax-Momo is somewhat more sensitive to the choice of F_* , but even the aggressive lower bound of $F_* = 0.0$ achieves 3.61 loss, which is close to the 3.58 optimum achieved with $F_* = 3.2$.

Effect of Stale Approximation. Table 1 shows the final validation loss and per-step wall-clock times of four methods (with tuned LRs) with and without stale nuclear norm approximations. We see that in all cases, the stale approximation increases the loss by at most 0.004, while sometimes even decreasing it. We therefore conclude that this approximation does not noticeably affect the final loss for these tuned algorithms, although it does afford a speedup; for MuonMax-Momo, the additional wall-clock time compared to MuonAdam is reduced from 11% to 5%.

6. Limitations and Future Work

A central limitation of our work is the lack of an explanation of when one norm yields a faster steepest descent algorithm than another, as we have focused on algorithm design and evaluation rather than explanatory theory. In some ways, our framework lends itself well to theory: all of the RSD variants with non-adaptive norms fit into the standard template of non-Euclidean steepest descent with respect to some norm. Thus, for these methods, one could import existing convergence theory for non-Euclidean steepest descent (Kelner et al., 2014; Kovalev, 2025; Shen et al., 2025; Sato et al., 2025; Sfyraiki & Wang, 2025; Xie et al., 2026; Riabinin et al., 2026; Tao et al., 2026).

In all of these analyses, the dual norm of the gradient plays an essential role, and one could compare different choices of product norms by inspecting how they affect this dual norm. Furthermore, almost all analyses of Muon have been in the smooth setting; indeed, Muon does not converge in the nonsmooth Lipschitz setting (Parshakova et al., 2026). The smoothness assumption itself depends on the choice of norm, so changing the product norm also changes the

corresponding smoothness constant. Thus, to understand the benefit of one norm over another, we need to compare not only the dual norm of the gradient, but the ratio between the squared dual norm and the corresponding smoothness constant (Davis & Drusvyatskiy, 2025).

In fact, the general question of determining when and why some norms yield faster steepest descent algorithms than others is a major open problem in this line of research. Answering this question would shed light on why SignGD/Adam outperform SGD, and why Muon outperforms both. Given the success of Muon, we view this question as an important future direction for deep learning optimization.

Another limitation of our Momo variants is the need to choose a loss lower bound F_* to run the algorithm. When the true value of F_* is unknown, as in language modeling, there are several options. The simplest is to set $F_* = 0$, which is a conservative choice and does not appear to greatly affect performance; see Figure 4. To squeeze out more performance, one could choose F_* using a scaling law: train on several smaller problem instances, with fewer parameters and tokens, fit a power law for the minimum achievable loss, and extrapolate this value, with some buffer, to the larger problem instance.

Acknowledgements

Michael Crawshaw conducted this work as an intern at the Flatiron Institute, and was also supported by the Doctoral Research Scholarship of George Mason University during the completion of this work. Mingrui Liu is partially supported by NSF award #2436217, #2425687.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Amsel, N., Persson, D., Musco, C., and Gower, R. M. The polar express: Optimal matrix sign methods and their application to the muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.
- Asi, H. and Duchi, J. C. Stochastic (approximate) proximal point methods: convergence, optimality, and adaptivity. *SIAM J. Optim.*, 29(3):2257–2290, 2019a. ISSN 1052-6234. doi: 10.1137/18M1230323.
- Asi, H. and Duchi, J. C. The importance of better models in stochastic optimization. *Proceedings of the National Academy of Sciences*, 116(46):22924–22930, 2019b. doi: 10.1073/pnas.1908018116.
- Asi, H., Chadha, K., Cheng, G., and Duchi, J. C. Mini-batch stochastic approximate proximal point methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Balles, L. and Hennig, P. Dissecting adam: The sign, magnitude and variance of stochastic gradients. In *International Conference on Machine Learning*, pp. 404–413. PMLR, 2018.
- Bernstein, J. and Newhouse, L. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024a.
- Bernstein, J. and Newhouse, L. Modular duality in deep learning. *arXiv preprint arXiv:2410.21265*, 2024b.
- Carlson, D., Cevher, V., and Carin, L. Stochastic Spectral Descent for Restricted Boltzmann Machines. In *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*, volume 38 of *Proceedings of Machine Learning Research*, pp. 111–119. PMLR, 09–12 May 2015a.
- Carlson, D. E., Collins, E., Hsieh, Y.-P., Carin, L., and Cevher, V. Preconditioned spectral descent for deep learning. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015b.
- Chen, K., Cutkosky, A., and Orabona, F. Implicit parameter-free online learning with truncated linear models. In *Proceedings of The 33rd International Conference on Algorithmic Learning Theory*, volume 167 of *Proceedings of Machine Learning Research*, pp. 148–175. PMLR, 29 Mar–01 Apr 2022.
- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., et al. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233, 2023.
- Davis, D. and Drusvyatskiy, D. Stochastic model-based minimization of weakly convex functions. *SIAM J. Optim.*, 29(1):207–239, 2019. ISSN 1052-6234. doi: 10.1137/18M1178244.
- Davis, D. and Drusvyatskiy, D. When do spectral gradient updates help in deep learning? *arXiv preprint arXiv:2512.04299*, 2025.
- Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., Zoph, B., Fedus, L., Bosma, M. P., Zhou, Z., Wang, T., Wang, E., Webster, K., Pellat, M., Robinson, K., Meier-Hellstern, K., Duke, T., Dixon, L., Zhang, K., Le, Q., Wu, Y., Chen, Z., and Cui, C. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 5547–5569. PMLR, 17–23 Jul 2022.
- Fan, C., Schmidt, M., and Thrampoulidis, C. Implicit bias of spectral descent and muon on multiclass separable data. *arXiv preprint arXiv:2502.04664*, 2025.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Jordan, K. cifar10-airbench, 2024. URL <https://github.com/KellerJordan/cifar10-airbench>.
- Jordan, K., Bernstein, J., Rappazzo, B., @fern-bear.bsky.social, Vlado, B., Jiacheng, Y., Cesista, F., Koszarsky, B., and @Grad62304977. modded-nanogpt: Speedrunning the nanogpt baseline, 2024a. URL <https://github.com/KellerJordan/modded-nanogpt>.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024b. URL <https://kellerjordan.github.io/posts/muon/>.
- Kelner, J. A., Lee, Y. T., Orecchia, L., and Sidford, A. An almost-linear-time algorithm for approximate max flow in undirected graphs, and its multicommodity generalizations. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*, pp. 217–226. SIAM, 2014.
- Kovalev, D. Understanding gradient orthogonalization for deep learning via non-euclidean trust-region optimization. *arXiv preprint arXiv:2503.12645*, 2025.

- Large, T., Liu, Y., Huh, M., Bahng, H., Isola, P., and Bernstein, J. Scalable optimization in the modular norm. *Advances in Neural Information Processing Systems*, 37: 73501–73548, 2024.
- Lau, T. T.-K. and Su, W. Symmetry-compatible principle for optimizer design: Embeddings, lm heads, swiglu mlps, and moe routers. *arXiv preprint arXiv:2605.18106*, 2026.
- Lau, T. T.-K., Long, Q., and Su, W. Polargrad: A class of matrix-gradient optimizers from a unifying preconditioning perspective. *arXiv preprint arXiv:2505.21799*, 2025.
- Li, J. and Hong, M. A note on the convergence of muon. *arXiv preprint arXiv:2502.02900*, 2025.
- Liu, H., Li, Z., Hall, D. L. W., Liang, P., and Ma, T. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations*, 2024.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- Loizou, N., Vaswani, S., Hadj Laradji, I., and Lacoste-Julien, S. Stochastic Polyak step-size for SGD: An adaptive learning rate for fast convergence. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pp. 1306–1314. PMLR, 13–15 Apr 2021.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Meng, S. Y. and Gower, R. M. A model-based method for minimizing CVaR and beyond. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 24436–24456. PMLR, 23–29 Jul 2023.
- Orvieto, A. and Gower, R. M. In search of adam’s secret sauce. In *Advances in Neural Information Processing Systems*, 2025.
- Parshakova, T., Khaled, A., Crawshaw, M., Garrigos, G., and Gower, R. M. Muon does not converge on convex lipschitz functions. *arXiv preprint arXiv:2605.08980*, 2026.
- Penedo, G., Kydlíček, H., Lozhkov, A., Mitchell, M., Raffel, C. A., Von Werra, L., Wolf, T., et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37: 30811–30849, 2024.
- Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., and Cevher, V. Training deep learning models with norm-constrained LMOs. In *Forty-second International Conference on Machine Learning*, 2025.
- Riabini, A., Shulgin, E., Grutkowska, K., and Richtárik, P. Gluon: Making muon & scion great again!(bridging theory and practice of lmo-based optimizers for llms). *arXiv preprint arXiv:2505.13416*, 2025.
- Riabini, A., Veprikov, A., Bolatov, A., Takáč, M., and Beznosikov, A. Where does warm-up come from? adaptive scheduling for norm-constrained optimizers. *arXiv preprint arXiv:2602.05813*, 2026.
- Sato, N., Naganuma, H., and Iiduka, H. Convergence bound and critical batch size of Muon optimizer. *arXiv preprint arXiv:2507.01598*, 2025.
- Schaipp, F., Gower, R. M., and Ulbrich, M. A stochastic proximal Polyak step size. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. Reproducibility Certification.
- Schaipp, F., Ohana, R., Eickenberg, M., Defazio, A., and Gower, R. M. Momo: momentum models for adaptive learning rates. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 43542–43570, 2024.
- Sfyraiki, M.-E. and Wang, J.-K. Lions and Muons: Optimization via stochastic Frank–Wolfe. *arXiv preprint arXiv:2506.04192*, 2025.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Shen, W., Huang, R., Huang, M., Shen, C., and Zhang, J. On the convergence analysis of Muon. *arXiv preprint arXiv:2505.23737*, 2025.
- Soboleva, D., Al-Khateeb, F., Myers, R., Steeves, J. R., Hestness, J., and Dey, N. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama, 2023. URL <https://huggingface.co/datasets/cerebras/SlimPajama-627B>.
- Su, W. Isotropic curvature model for understanding deep learning optimization: Is gradient orthogonalization optimal? *arXiv preprint arXiv:2511.00674*, 2025.
- Tao, H., Yu, D., and Zhang, L. When and why signsgd outperforms sgd: A theoretical study based on ℓ_1 -norm lower bounds. *arXiv preprint arXiv:2605.06615*, 2026.

Tsilivis, N., Vardi, G., and Kempe, J. Flavors of margin: Implicit bias of steepest descent in homogeneous neural networks. In *The Thirteenth International Conference on Learning Representations*, 2025.

Xie, S., Wang, T., Wu, B., and Li, Z. A tale of two geometries: Adaptive optimizers and Non-Euclidean descent. In *The Fourteenth International Conference on Learning Representations*, 2026.

Xie, X., Zhou, P., Li, H., Lin, Z., and Yan, S. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):9508–9520, 2024.

Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Tang, J., Wang, J., Yang, J., Tu, J., Zhang, J., Ma, J., Yang, J., Xu, J., Zhou, J., Bai, J., He, J., Lin, J., Dang, K., Lu, K., Chen, K., Yang, K., Li, M., Xue, M., Ni, N., Zhang, P., Wang, P., Peng, R., Men, R., Gao, R., Lin, R., Wang, S., Bai, S., Tan, S., Zhu, T., Li, T., Liu, T., Ge, W., Deng, X., Zhou, X., Ren, X., Zhang, X., Wei, X., Ren, X., Liu, X., Fan, Y., Yao, Y., Zhang, Y., Wan, Y., Chu, Y., Liu, Y., Cui, Z., Zhang, Z., Guo, Z., and Fan, Z. Qwen2 technical report. *eprint arXiv: 2407.10671*, 2024.

Contents

1	Introduction	1
2	Related Work	2
3	Steepest Descent on Neural Networks	3
3.1	Constrained vs Regularized Steepest Descent	3
3.2	Product Norms	3
3.3	Incorporating Adam	4
3.4	The Whole Framework	4
4	Model Truncation	6
5	Experiments	6
5.1	FineWeb Dataset	7
5.2	SlimPajama Dataset	8
5.3	Ablations	8
6	Limitations and Future Work	9
A	Proofs from Section 3	14
A.1	Recovering Existing Algorithms	18
B	Proofs from Section 4	20
C	Experimental Details for Language Modeling	25
D	Additional Language Modeling Results	26
D.1	FineWeb	26
D.1.1	Additional Baselines	28
D.1.2	Additional Metrics	30
D.1.3	Varying Batch Size	30
D.1.4	Qwen2Moe model	32
D.2	SlimPajama	33
D.2.1	Additional Metrics	33
E	Image Classification Experiments	34

A. Proofs from Section 3

In what follows, for a norm denoted by a subscript such as $\|\cdot\|_\infty$, we will sometimes replace $\text{LMO}_{\|\cdot\|_\infty}$ with LMO_∞ .

Proposition 3.1. [Constrained Steepest Descent] The CSD update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\|\mathbf{w} - \mathbf{w}_t\| \leq \eta} \{F(\mathbf{w}_t) + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle\} \\ &= \mathbf{w}_t + \eta \text{LMO}(\mathbf{m}_t). \end{aligned} \quad (2)$$

Proof of Proposition 3.1. Denoting $r = \|\mathbf{w} - \mathbf{w}_t\|$ and $\Delta = (\mathbf{w} - \mathbf{w}_t)/\|\mathbf{w} - \mathbf{w}_t\|$, we can change variables in the optimization problem from Equation 2, yielding $\mathbf{w}_{t+1} = \mathbf{w}_t + r_t \Delta_t$, where

$$(r_t, \Delta_t) = \arg \min_{r \in [0, \eta], \|\Delta\|=1} \{r \langle \mathbf{m}_t, \Delta \rangle\},$$

which can be separated into

$$\Delta_t = \arg \min_{\|\Delta\|=1} \langle \mathbf{m}_t, \Delta \rangle = \text{LMO}(\mathbf{m}_t),$$

and

$$r_t = \arg \min_{r \in [0, \eta]} \{r \langle \mathbf{m}_t, \Delta_t \rangle\} = \arg \min_{r \in [0, \eta]} \{r \langle \mathbf{m}_t, \text{LMO}(\mathbf{m}_t) \rangle\} = \arg \min_{r \in [0, \eta]} \{-r \|\mathbf{m}_t\|_*\} = \eta,$$

so $\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \text{LMO}(\mathbf{m}_t)$. □

Proposition 3.2. [Regularized Steepest Descent] The RSD update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\mathbf{w}} \left\{ F(\mathbf{w}_t) + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle \right. \\ &\quad \left. + \frac{1}{2\eta} \|\mathbf{w} - \mathbf{w}_t\|^2 \right\} \\ &= \mathbf{w}_t + \eta \|\mathbf{m}_t\|_* \text{LMO}(\mathbf{m}_t) \end{aligned} \quad (3)$$

Proof of Proposition 3.2. For the optimization problem from Equation 3, we use the same change of variables as in the proof of Proposition 3.1: $r = \|\mathbf{w} - \mathbf{w}_t\|$ and $\Delta = (\mathbf{w} - \mathbf{w}_t)/\|\mathbf{w} - \mathbf{w}_t\|$. Therefore $\mathbf{w}_{t+1} = \mathbf{w}_t + r_t \Delta_t$, where

$$(r_t, \Delta_t) = \arg \min_{r \geq 0, \|\Delta\|=1} \left\{ r \langle \mathbf{m}_t, \Delta \rangle + \frac{r^2}{2\eta} \right\},$$

which can be separated into

$$\Delta_t = \arg \min_{\|\Delta\|=1} \langle \mathbf{m}_t, \Delta \rangle = \text{LMO}(\mathbf{m}_t),$$

and

$$\begin{aligned} r_t &= \arg \min_{r \geq 0} \left\{ r \langle \mathbf{m}_t, \Delta_t \rangle + \frac{r^2}{2\eta} \right\} \\ &= \arg \min_{r \geq 0} \left\{ r \langle \mathbf{m}_t, \text{LMO}(\mathbf{m}_t) \rangle + \frac{r^2}{2\eta} \right\} \\ &= \arg \min_{r \geq 0} \left\{ -r \|\mathbf{m}_t\|_* + \frac{r^2}{2\eta} \right\} \\ &= \eta \|\mathbf{m}_t\|_*, \end{aligned}$$

so that $\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \|\mathbf{m}_t\|_* \text{LMO}(\mathbf{m}_t)$. □

Lemma 3.3. [LMO and Dual of Product Norms] For each $i \in [n]$, let g_i be a norm on $\mathbb{R}^{d_i}$, let f be a norm on $\mathbb{R}^n$, and denote their dual norms as g_i^* and f_* . Then the product norm $h : \mathbb{R}^{d_1} \times \dots \times \mathbb{R}^{d_n} \rightarrow \mathbb{R}$ defined by

$$h(\mathbf{w}^1, \dots, \mathbf{w}^n) = f(g_1(\mathbf{w}^1), \dots, g_n(\mathbf{w}^n))$$

is indeed a norm, and its LMO and dual are given by

$$\begin{aligned} \text{LMO}_h(\mathbf{w}^1, \dots, \mathbf{w}^n) &= (\phi_1 \text{LMO}_{g_1}(\mathbf{w}^1), \dots, \phi_n \text{LMO}_{g_n}(\mathbf{w}^n)) \\ h_*(\mathbf{w}^1, \dots, \mathbf{w}^n) &= f_*(g_1^*(\mathbf{w}^1), \dots, g_n^*(\mathbf{w}^n)), \end{aligned}$$

where $(\phi_1, \dots, \phi_n) := -\text{LMO}_f(g_1^*(\mathbf{w}^1), \dots, g_n^*(\mathbf{w}^n))$.

Proof of Lemma 3.3. To show that h is a norm, we only need to show that

1. $h(\mathbf{w}_1, \dots, \mathbf{w}_n) \geq 0$ for all $\mathbf{w}_1, \dots, \mathbf{w}_n$,
2. $h(\mathbf{w}_1, \dots, \mathbf{w}_n) = 0$ if and only if $(\mathbf{w}_1, \dots, \mathbf{w}_n) = \mathbf{0}$,
3. $h(\lambda \mathbf{w}_1, \dots, \lambda \mathbf{w}_n) = |\lambda| h(\mathbf{w}_1, \dots, \mathbf{w}_n)$ for all $\lambda \in \mathbb{R}, \mathbf{w}_1, \dots, \mathbf{w}_n$,
4. $h(\mathbf{w}_1 + \mathbf{v}_1, \dots, \mathbf{w}_n + \mathbf{v}_n) \leq h(\mathbf{w}_1, \dots, \mathbf{w}_n) + h(\mathbf{v}_1, \dots, \mathbf{v}_n)$ for all $\mathbf{w}_1, \mathbf{v}_1, \dots, \mathbf{w}_n, \mathbf{v}_n$.

All of these properties hold immediately from the definition of $h = f \circ (g_1, \dots, g_n)$ together with repeated applications of the norm properties of f and $g_1, \dots, g_n$.

From the definition of the dual norm,

$$\begin{aligned} h_*(\mathbf{w}_1, \dots, \mathbf{w}_n) &= \max \left\{ \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{v}_i \rangle \mid h(\mathbf{v}_1, \dots, \mathbf{v}_n) = 1 \right\} \\ &= \max \left\{ \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{v}_i \rangle \mid f(g_1(\mathbf{v}_1), \dots, g_n(\mathbf{v}_n)) = 1 \right\}. \end{aligned} \quad (10)$$

We use a change of variables $\mathbf{u}_i = \mathbf{v}_i / g_i(\mathbf{v}_i)$ and $r_i = g_i(\mathbf{v}_i)$, which separates the update direction $\mathbf{u}_i$ (with unit norm) from the update norm r_i . So Equation 10 is equivalent to

$$h_*(\mathbf{w}_1, \dots, \mathbf{w}_n) = \max \left\{ \sum_{i=1}^n r_i \langle \mathbf{w}_i, \mathbf{u}_i \rangle \mid f(r_1, \dots, r_n) = 1 \right\} \quad (11)$$

Note that the condition $f(r_1, \dots, r_n) = 1$ does not involve $\mathbf{u}_i$, so each term $r_i \langle \mathbf{w}_i, \mathbf{u}_i \rangle$ is maximized when

$$\mathbf{u}_i = \arg \max_{g_i(\mathbf{z}_i)=1} \langle \mathbf{w}_i, \mathbf{z}_i \rangle = -\text{LMO}_{g_i}(\mathbf{w}_i),$$

with maximum value $\langle \mathbf{w}_i, \mathbf{u}_i \rangle = g_{i,*}(\mathbf{w}_i)$. Using this in Equation 11 gives

$$h_*(\mathbf{w}_1, \dots, \mathbf{w}_n) = \max \left\{ \sum_{i=1}^n r_i g_{i,*}(\mathbf{w}_i) \mid f(r_1, \dots, r_n) = 1 \right\}.$$

Denoting $\mathbf{r} = (r_1, \dots, r_n)$ and $\mathbf{s} = (g_{1,*}(\mathbf{w}_1), \dots, g_{n,*}(\mathbf{w}_n))$, this is equivalent to

$$\begin{aligned} h_*(\mathbf{w}_1, \dots, \mathbf{w}_n) &= \max \{ \langle \mathbf{r}, \mathbf{s} \rangle \mid f(\mathbf{r}) = 1 \} \\ &= f_*(\mathbf{s}), \end{aligned}$$

which gives us the dual norm h_* .

To obtain LMO_h , we only need to look at the value of the variables which achieved the maximum in the above derivation:

$$\mathbf{u}_i = -\text{LMO}_{g_i}(\mathbf{w}_i), \quad \text{and} \quad \mathbf{r} = \text{LMO}_f(g_{1,*}(\mathbf{w}_1), \dots, g_{n,*}(\mathbf{w}_n))$$

so that

$$\mathbf{v}_i = r_i \text{LMO}_f(g_{1,*}(\mathbf{w}_1), \dots, g_{n,*}(\mathbf{w}_n))$$

maximizes $\sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{v}_i \rangle$ subject to $h(\mathbf{v}_1, \dots, \mathbf{v}_n) = 1$. Note that $\text{LMO}_h(\mathbf{w}_1, \dots, \mathbf{w}_n)$ is exactly the minimizer of $\sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{v}_i \rangle$ subject to the same norm constraint; since $\sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{v}_i \rangle$ is linear in $\mathbf{v}_i$, the minimizer is the negative of the maximizer. Therefore

$$\text{LMO}_h(\mathbf{w}_1, \dots, \mathbf{w}_n) = -(r_1 \text{LMO}_{g_1}(\mathbf{w}_1), \dots, r_n \text{LMO}_{g_n}(\mathbf{w}_n)).$$

□

The following lemma will be useful later for quickly computing duals and LMOs of various norms.

Lemma A.1. For any norm $\|\cdot\|$ on $\mathbb{R}^d$ full rank matrix $\mathbf{D} \in \mathbb{R}^{d \times d}$, the norm defined by $\|\mathbf{v}\|_{\mathbf{D}} = \|\mathbf{D}\mathbf{v}\|$ has

$$\begin{aligned} \text{LMO}_{\|\cdot\|_{\mathbf{D}}}(\mathbf{v}) &= \mathbf{D}^{-1} \text{LMO}_{\|\cdot\|}(\mathbf{D}^{-T} \mathbf{v}), \\ \|\mathbf{v}\|_{\mathbf{D},*} &= \|\mathbf{D}^{-T} \mathbf{v}\|_*. \end{aligned}$$

Proof. The fact that $\|\cdot\|_{\mathbf{D}}$ is a norm follows immediately from the norm properties of $\|\cdot\|$ together with the fact that $\mathbf{D}$ is full rank. For the dual norm,

$$\|\mathbf{v}\|_{\mathbf{D},*} = \max_{\|\mathbf{u}\|_{\mathbf{D}}=1} \langle \mathbf{v}, \mathbf{u} \rangle = \max_{\|\mathbf{D}\mathbf{u}\|=1} \langle \mathbf{v}, \mathbf{u} \rangle$$

and a change of variables $\mathbf{z} = \mathbf{D}\mathbf{u}$ yields

$$\|\mathbf{v}\|_{\mathbf{D},*} = \max_{\|\mathbf{z}\|=1} \langle \mathbf{v}, \mathbf{D}^{-1} \mathbf{z} \rangle = \max_{\|\mathbf{z}\|=1} \langle \mathbf{D}^{-T} \mathbf{v}, \mathbf{z} \rangle = \|\mathbf{D}^{-T} \mathbf{v}\|_*.$$

For the LMO, we can look at the value of the variables that maximize the inner product in the above:

$$\mathbf{z} = \arg \max_{\|\mathbf{z}\|=1} \langle \mathbf{D}^{-T} \mathbf{v}, \mathbf{z} \rangle = -\text{LMO}_{\|\cdot\|}(\mathbf{D}^{-T} \mathbf{v}).$$

Returning to the $\mathbf{u}$ variable then gives

$$\mathbf{u} = \mathbf{D}^{-1} \mathbf{z} = -\mathbf{D}^{-1} \text{LMO}_{\|\cdot\|}(\mathbf{D}^{-T} \mathbf{v})$$

which maximizes $\langle \mathbf{v}, \mathbf{u} \rangle$ subject to $\|\mathbf{u}\|_{\mathbf{D}} = 1$. Since $\langle \mathbf{v}, \mathbf{u} \rangle$ is linear in $\mathbf{u}$, the minimizer of $\langle \mathbf{v}, \mathbf{u} \rangle$ under the norm constraint $\|\mathbf{u}\|_{\mathbf{D}} = 1$ is exactly the negative of the maximizer under the same constraint. So

$$\text{LMO}_{\|\cdot\|_{\mathbf{D}}}(\mathbf{v}) = \arg \min_{\|\mathbf{u}\|_{\mathbf{D}}=1} \langle \mathbf{v}, \mathbf{u} \rangle = \mathbf{D}^{-1} \text{LMO}_{\|\cdot\|}(\mathbf{D}^{-T} \mathbf{v})$$

□

Proposition 3.4. The t -th update of Adam is the CSD with step size η with respect to the norm:

$$\|\boldsymbol{\theta}\|_{\text{ada}\infty} := \|\text{Diag}\left(\frac{\sqrt{v_t} + \epsilon}{|m_t|}\right) \boldsymbol{\theta}\|_{\infty}$$

Proof of Proposition 3.4. Let $\mathbf{D} = \text{Diag}\left(\frac{\sqrt{v_t} + \epsilon}{|m_t|}\right)$, so that $\|\boldsymbol{\theta}\|_{\text{ada}\infty} = \|\mathbf{D}\boldsymbol{\theta}\|_{\infty}$. Then by Proposition 3.1, one step of CSD

Algorithm 1 MuonAdam: where $W^1, \dots, W^L$ are the weight matrices, and θ are all other parameters flattened into a vector.

Inputs: $W_0^1, \dots, W_0^L, \theta_0$, learning rates η_b, η_m , EMA parameters β, β_1, β_2

```

1: for  $t = 0, 1, \dots, T - 1$  do
2:    $(G_t^1, \dots, G_t^L, g_t^\theta) \leftarrow \text{backward}(W_t^1, \dots, W_t^L, \theta_t)$ 
3:   for  $\ell = 1, \dots, L$  do
4:      $M_t^\ell = \beta M_{t-1}^\ell + (1 - \beta) G_t^\ell$ 
5:      $W_{t+1}^\ell \leftarrow W_t^\ell - \eta_m \text{polar}(M_t^\ell)$ 
6:   end for
7:    $m_t^\theta = \beta_1 m_{t-1}^\theta + (1 - \beta_1) g_t^\theta$ 
8:    $v_t^\theta = \beta_2 v_{t-1}^\theta + (1 - \beta_2) g_t^\theta \odot g_t^\theta$ 
9:    $\theta_{t+1} = \theta_t - \eta_b \frac{m_t^\theta}{\sqrt{v_t^\theta + \epsilon}}$ 
10: end for
    
```

w.r.t. $\|\cdot\|_{\text{ada}\infty}$ is given by

$$\begin{aligned}
 \theta_{t+1} &= \theta_t + \eta \text{LMO}_{\text{ada}\infty}(m_t) \\
 &\stackrel{(i)}{=} \theta_t + \eta D^{-1} \text{LMO}_\infty(D^{-T} m_t) \\
 &\stackrel{(ii)}{=} \theta_t - \eta D^{-1} \text{sign}(D^{-T} m_t) \\
 &= \theta_t - \eta \text{Diag}\left(\frac{|m_t|}{\sqrt{v_t} + \epsilon}\right) \text{sign}\left(\text{Diag}\left(\frac{|m_t|}{\sqrt{v_t} + \epsilon}\right) m_t\right) \\
 &= \theta_t - \eta \frac{|m_t|}{\sqrt{v_t} + \epsilon} \odot \text{sign}(m_t) \\
 &= \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon},
 \end{aligned}$$

where (i) uses Lemma A.1 and (ii) uses $\text{LMO}_\infty(v) = -\text{sign}(v)$. $\square$

Proposition 3.5. The t -th update of Adam is the RSD with step size η with respect to the norm:

$$\|\theta\|_{\text{ada}2} := \sqrt{\langle \text{Diag}(\sqrt{v_t} + \epsilon) \theta, \theta \rangle}$$

Proof of Proposition 3.5. Let $D = \text{Diag}(\sqrt{v_t} + \epsilon)$, so that $\|\theta\|_{\text{ada}2} = \|D\theta\|_2$. Then by Proposition 3.2, one step of RSD w.r.t. $\|\cdot\|_{\text{ada}2}$ is given by

$$\begin{aligned}
 \theta_{t+1} &= \theta_t + \eta \|m_t\|_{\text{ada}2,*} \text{LMO}_{\text{ada}2}(m_t) \\
 &\stackrel{(i)}{=} \theta_t + \eta \|D^{-T} m_t\|_{2,*} D^{-1} \text{LMO}_2(D^{-T} m_t) \\
 &\stackrel{(ii)}{=} \theta_t - \eta \|D^{-T} m_t\|_2 D^{-1} \frac{D^{-T} m_t}{\|D^{-T} m_t\|_2} \\
 &= \theta_t - \eta D^{-1} D^{-T} m_t \\
 &= \theta_t - \eta \text{Diag}\left(\frac{1}{\sqrt{v_t} + \epsilon}\right) m_t \\
 &= \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon},
 \end{aligned}$$

where (i) uses Lemma A.1 and (ii) uses $\text{LMO}_2(v) = -v/\|v\|_2$. $\square$

For reference, we include the pseudocode for MuonAdam (Muon side-by-side with Adam) in Algorithm 1.

Proposition 3.6. MuonAdam (Algorithm 1) is exactly CSD with step size η_m with respect to

$$\|\mathbf{W}\|_{\text{muon}} = \max \left(\max_{\ell \in [L]} \|\mathbf{W}^\ell\|_{2 \rightarrow 2}, \frac{\eta_m}{\eta_b} \|\boldsymbol{\theta}\|_{\text{ada}\infty} \right)$$

Proof of Proposition 3.6. By Proposition 3.1, one step of CSD w.r.t. $\|\cdot\|_{\text{muon}}$ can be written as

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \eta_m \text{LMO}_{\text{muon}}(\mathbf{M}_t), \quad (12)$$

where $\mathbf{M}_t$ is the momentum buffer for all network parameters, i.e. it is the concatenation of the momentum buffers of each parameter:

$$\mathbf{M}_t = (\mathbf{M}_t^1, \dots, \mathbf{M}_t^L, \mathbf{m}_t^\theta).$$

Denote $\lambda = \eta_b/\eta_m$. To compute the LMO term, we can rewrite $\|\mathbf{W}\|_{\text{muon}}$ as

$$\|\mathbf{W}\|_{\text{muon}} = \max \left(\|\mathbf{W}^1\|_{2 \rightarrow 2}, \dots, \|\mathbf{W}^L\|_{2 \rightarrow 2}, \frac{1}{\lambda} \|\boldsymbol{\theta}\|_{\text{ada}\infty} \right),$$

so that $\|\cdot\|_{\text{muon}}$ can be written as the composition (as in Lemma 3.3)

$$\|\mathbf{W}\|_{\text{muon}} = f(g_1(\mathbf{W}^1), \dots, g_L(\mathbf{W}^L), g_{L+1}(\boldsymbol{\theta})),$$

with $g_i(\mathbf{M}) = \|\mathbf{M}\|_{2 \rightarrow 2}$ for $i \in [L]$, $g_{L+1}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_{\text{ada}\infty}$, and $f(\mathbf{v}) = \|\mathbf{D}\mathbf{v}\|_\infty$, where $\mathbf{D} = \text{Diag}(1, \dots, 1, 1/\lambda) \in \mathbb{R}^{(L+1) \times (L+1)}$. Therefore, by Lemma 3.3, the update in Equation 12 is equivalent to

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell + \eta_m \phi_\ell \text{LMO}_{2 \rightarrow 2}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \eta_m \phi_{L+1} \text{LMO}_{\text{ada}\infty}(\mathbf{m}_t^\theta), \end{aligned} \quad (13)$$

where $\phi = -\text{LMO}_f(\|\mathbf{M}_t^1\|_{\text{nuc}}, \dots, \|\mathbf{M}_t^L\|_{\text{nuc}}, \|\mathbf{m}_t^\theta\|_{\text{ada}\infty}, *)$. We know $\text{LMO}_{2 \rightarrow 2}(\mathbf{M}) = -\text{polar}(\mathbf{M})$, and we proved in Proposition 3.4 that

$$\text{LMO}_{\text{ada}\infty}(\mathbf{v}) = -\frac{|\mathbf{m}_t^\theta|}{\sqrt{\mathbf{v}_t^\theta + \epsilon}} \text{sign}(\mathbf{v}),$$

so the LMO terms in Equation 13 can be simplified as

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \phi_\ell \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_m \phi_{L+1} \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}}. \end{aligned} \quad (14)$$

To simplify ϕ , we use Lemma A.1. Denoting $\mathbf{u} = (\|\mathbf{M}_t^1\|_{\text{nuc}}, \dots, \|\mathbf{M}_t^L\|_{\text{nuc}}, \|\mathbf{m}_t^\theta\|_{\text{ada}\infty}, *)$, we have

$$\phi = -\text{LMO}_f(\mathbf{u}) = -\mathbf{D}^{-1} \text{LMO}_\infty(\mathbf{D}^{-T} \mathbf{u}) = \mathbf{D}^{-1} \text{sign}(\mathbf{D}^{-T} \mathbf{u}) = \mathbf{D}^{-1} \mathbf{1},$$

so that $\phi_\ell = 1$ for $\ell \in [L]$ and $\phi_{L+1} = \lambda = \eta_b/\eta_m$. Plugging back to Equation 14 gives

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_a \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}}, \end{aligned}$$

which is exactly the update from Algorithm 1. □

A.1. Recovering Existing Algorithms

Propositions A.2 and A.3 below show how Scion (Pethick et al., 2025) and PolarGrad (Lau et al., 2025) are both instances of our steepest descent framework. All notation in this section follows that of Section 3.

Throughout our paper, Scion refers to the following algorithm:

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_b \text{sign}(\mathbf{m}_t^\theta). \end{aligned} \quad (15)$$

This update differs slightly from the algorithm proposed by Pethick et al. (2025) in that for each parameter matrix $\mathbf{W}$ of shape $d_{\text{out}} \times d_{\text{in}}$, we omit a coefficient of $\sqrt{d_{\text{out}}/d_{\text{in}}}$ from the update. This corresponds to assigning to each weight matrix the spectral norm $\|\cdot\|_{2 \rightarrow 2}$ rather than the RMS to RMS operator norm used by Pethick et al. (2025). Indeed, the motivation of the RMS to RMS norm is to allow for hyperparameter transfer across architecture sizes, but in our work we focus on LR sensitivity for a fixed architecture, so for simplicity we did not employ this RMS scaling. However, we could easily recover the RMS variant by replacing the spectral norm $\|\cdot\|_{2 \rightarrow 2}$ with the RMS to RMS operator norm.

Proposition A.2. Scion is exactly CSD with step size η_m with respect to

$$\|\mathbf{W}\|_{\text{scion}} = \max \left(\max_{1 \leq \ell \leq L} \|\mathbf{W}^\ell\|_{2 \rightarrow 2}, \frac{\eta_m}{\eta_b} \|\boldsymbol{\theta}\|_\infty \right).$$

Note that the same conclusion was already reached by Pethick et al. (2025), that is, they already described Scion in terms of a norm on the space of all parameters (see their Equation (6)). We include Proposition A.2 to specify how Scion is a special case of our framework.

Proof. The proof is very similar to that of Proposition 3.6, since Muon-Adam differs from Scion only in that Adam is used for non-matrix parameters instead of sign SGD with momentum.

By Proposition 3.1, one step of CSD w.r.t. $\|\cdot\|_{\text{scion}}$ can be written as

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \eta_m \text{LMO}_{\text{scion}}(\mathbf{M}_t), \quad (16)$$

where $\mathbf{M}_t$ is the momentum buffer for all network parameters, i.e. it is the concatenation of the momentum buffers of each parameter:

$$\mathbf{M}_t = (\mathbf{M}_t^1, \dots, \mathbf{M}_t^L, \mathbf{m}_t^\theta).$$

Denote $\lambda = \eta_b/\eta_m$. To compute the LMO term, we can rewrite $\|\mathbf{W}\|_{\text{scion}}$ as

$$\|\mathbf{W}\|_{\text{scion}} = \max \left(\|\mathbf{W}^1\|_{2 \rightarrow 2}, \dots, \|\mathbf{W}^L\|_{2 \rightarrow 2}, \frac{1}{\lambda} \|\boldsymbol{\theta}\|_\infty \right),$$

so that $\|\cdot\|_{\text{scion}}$ can be written as the composition (as in Lemma 3.3)

$$\|\mathbf{W}\|_{\text{scion}} = f(g_1(\mathbf{W}^1), \dots, g_L(\mathbf{W}^L), g_{L+1}(\boldsymbol{\theta})),$$

with $g_i(\mathbf{M}) = \|\mathbf{M}\|_{2 \rightarrow 2}$ for $i \in [L]$, $g_{L+1}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_\infty$, and $f(\mathbf{v}) = \|\mathbf{D}\mathbf{v}\|_\infty$, where $\mathbf{D} = \text{Diag}(1, \dots, 1, 1/\lambda) \in \mathbb{R}^{(L+1) \times (L+1)}$. Therefore, by Lemma 3.3, the update in Equation 16 is equivalent to

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell + \eta_m \phi_\ell \text{LMO}_{2 \rightarrow 2}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \eta_m \phi_{L+1} \text{LMO}_\infty(\mathbf{m}_t^\theta), \end{aligned} \quad (17)$$

where $\phi = -\text{LMO}_f(\|\mathbf{M}_t^1\|_{\text{nuc}}, \dots, \|\mathbf{M}_t^L\|_{\text{nuc}}, \|\mathbf{m}_t^\theta\|_{\text{ada}\infty,*})$. We know $\text{LMO}_{2 \rightarrow 2}(\mathbf{M}) = -\text{polar}(\mathbf{M})$ and $\text{LMO}_\infty(\mathbf{v}) = -\text{sign}(\mathbf{v})$, so the LMO terms in Equation 17 can be simplified as

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \phi_\ell \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_m \phi_{L+1} \text{sign}(\mathbf{m}_t^\theta). \end{aligned} \quad (18)$$

To simplify ϕ , we use Lemma A.1. Denoting $\mathbf{u} = (\|\mathbf{M}_t^1\|_{\text{nuc}}, \dots, \|\mathbf{M}_t^L\|_{\text{nuc}}, \|\mathbf{m}_t^\theta\|_1)$, we have

$$\phi = -\text{LMO}_f(\mathbf{u}) = -\mathbf{D}^{-1} \text{LMO}_\infty(\mathbf{D}^{-T} \mathbf{u}) = \mathbf{D}^{-1} \text{sign}(\mathbf{D}^{-T} \mathbf{u}) = \mathbf{D}^{-1} \mathbf{1},$$

so that $\phi_\ell = 1$ for $\ell \in [L]$ and $\phi_{L+1} = \lambda = \eta_b/\eta_m$. Plugging back to Equation 18 gives

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_a \text{sign}(\mathbf{m}_t^\theta), \end{aligned}$$

which is exactly the update from Scion (Equation 15). $\square$

Throughout our paper, PolarGrad refers to the following algorithm:

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_s \|\mathbf{M}_t^\ell\|_{\text{nuc}} \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_b \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta} + \epsilon}. \end{aligned} \quad (19)$$

Lau et al. (2025) use the name "PolarGrad" to refer to a class of matrix-aware optimization methods, whereas we use it to refer to the specific method called "Vanilla PolarGrad" by Lau et al. (2025) (see their Equation (8)), with Adam used for non-matrix parameters.

Proposition A.3. PolarGrad is exactly CSD with step size η_m with respect to

$$\|\mathbf{W}\|_{\text{PG}} = \sqrt{\sum_{\ell=1}^L \|\mathbf{W}^\ell\|_{2 \rightarrow 2}^2 + \frac{\eta_m}{\eta_b} \|\boldsymbol{\theta}\|_{\text{ada2}}^2}.$$

Proof. Denote $\lambda = \eta_b/\eta_m$. Notice that $\|\cdot\|_{\text{PG}}$ can be written as a composition (as in Lemma 3.3) as:

$$\|\mathbf{W}\|_{\text{PG}} = f(g_1(\mathbf{W}^1), \dots, g_L(\mathbf{W}^L), g_{L+1}(\boldsymbol{\theta})),$$

with $g_i(\mathbf{M}) = \|\mathbf{M}\|_{2 \rightarrow 2}$ for $i \leq L$, $g_{L+1}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_{\text{ada2}}/\sqrt{\lambda}$, and $f(\mathbf{v}) = \|\mathbf{v}\|_2$. Therefore, $\|\cdot\|_{\text{PG}}$ uses the ℓ_2 norm as the outer norm, so Equation 7 implies that the update can be rewritten as

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell + \eta_m \|\mathbf{M}_t^\ell\|_{\text{nuc}} \text{LMO}_{2 \rightarrow 2}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \lambda \eta_m \|\mathbf{m}_t^\theta\|_{\text{ada2},*} \text{LMO}_{\text{ada2}}(\mathbf{m}_t^\theta). \end{aligned} \quad (20)$$

The update to $\mathbf{W}_t^\ell$ can be simplified by plugging in $\text{LMO}_{2 \rightarrow 2}(\mathbf{M}) = -\text{polar}(\mathbf{M})$, and the update to $\boldsymbol{\theta}_t$ can be simplified by plugging in the definition of λ and the dual and LMO of $\|\cdot\|_{\text{ada2}}$ from Proposition 3.5. This yields that Equation 20 is equivalent to

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta_m \|\mathbf{M}_t^\ell\|_{\text{nuc}} \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta_b \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta} + \epsilon}, \end{aligned}$$

which is exactly PolarGrad (Equation 19). $\square$

B. Proofs from Section 4

Proposition 4.1. [Constrained Momo] The ball constrained truncated model update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\|\mathbf{w} - \mathbf{w}_t\| \leq \eta} \left\{ \max \left(\tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle, F_* \right) \right\} \\ &= \mathbf{w}_t + \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right) \text{LMO}(\mathbf{m}_t) \end{aligned}$$

Proof of Proposition 4.1. Similar to the proofs of Proposition 3.1 and 3.2, we change variables to parameterize the magnitude $r = \|\mathbf{w} - \mathbf{w}_t\|$ and direction $\boldsymbol{\Delta} = (\mathbf{w} - \mathbf{w}_t)/\|\mathbf{w} - \mathbf{w}_t\|$ of the update. So $\mathbf{w}_{t+1} = \mathbf{w}_t + r_t \boldsymbol{\Delta}_t$, where

$$(r_t, \boldsymbol{\Delta}_t) = \arg \min_{r \in [0, \eta], \|\boldsymbol{\Delta}\|=1} \left\{ \max \left(\tilde{F}_t + r \langle \mathbf{m}_t, \boldsymbol{\Delta} \rangle, F_* \right) \right\}.$$

Since $\max \left(\tilde{F}_t + r \langle \mathbf{m}_t, \boldsymbol{\Delta} \rangle, F_* \right)$ is monotonic in $\langle \mathbf{m}_t, \boldsymbol{\Delta} \rangle$,

$$\boldsymbol{\Delta}_t = \arg \min_{\|\boldsymbol{\Delta}\|=1} \langle \mathbf{m}_t, \boldsymbol{\Delta} \rangle = \text{LMO}(\mathbf{m}_t),$$

so

$$r_t = \arg \min_{r \in [0, \eta]} \left\{ \max \left(\tilde{F}_t - r \langle \mathbf{m}_t, \Delta_t \rangle, F_* \right) \right\} = \arg \min_{r \in [0, \eta]} \left\{ \max \left(\tilde{F}_t - r \|\mathbf{m}_t\|_*, F_* \right) \right\}.$$

Note that $\max \left(\tilde{F}_t - r \|\mathbf{m}_t\|_*, F_* \right)$ can have multiple minimizing values of $r \in [0, \eta]$. If $\eta \leq (\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_*$, then the minimizer $r = \eta$ is unique. If $\eta \geq (\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_*$, then any r with $(\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_* \leq r \leq \eta$ achieves the minimum F_* . In this case, we choose the value that minimizes the norm of the update, i.e. $r_t = (\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_*$. These two cases are summarized as:

$$r_t = \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right),$$

so

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right) \text{LMO}(\mathbf{m}_t).$$

□

Proposition 4.2. [Regularized Momo] The regularized truncated model update is given by

$$\begin{aligned} \mathbf{w}_{t+1} &= \arg \min_{\mathbf{w}} \left\{ \max \left(\tilde{F}_t + \langle \mathbf{m}_t, \mathbf{w} - \mathbf{w}_t \rangle, F_* \right) \right. \\ &\quad \left. + \frac{1}{2\eta} \|\mathbf{w} - \mathbf{w}_t\|^2 \right\} \\ &= \mathbf{w}_t + \min \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*^2} \right) \|\mathbf{m}_t\|_* \text{LMO}(\mathbf{m}_t) \end{aligned}$$

Proof of Proposition 4.2. Similar to the proofs of Proposition 3.1 and 3.2, we perform a change of variables to parameterize the magnitude $r = \|\mathbf{w} - \mathbf{w}_t\|$ and direction $\Delta = (\mathbf{w} - \mathbf{w}_t) / \|\mathbf{w} - \mathbf{w}_t\|$ of the update. So $\mathbf{w}_{t+1} = \mathbf{w}_t + r_t \Delta_t$, where

$$(r_t, \Delta_t) = \arg \min_{r \geq 0, \|\Delta\|=1} \left\{ \max \left(\tilde{F}_t + r \langle \mathbf{m}_t, \Delta \rangle, F_* \right) + \frac{r^2}{2\eta} \right\}.$$

Note that $\max \left(\tilde{F}_t + r \langle \mathbf{m}_t, \Delta \rangle, F_* \right) + \frac{r^2}{2\eta}$ is monotonic in $\langle \mathbf{m}_t, \Delta \rangle$, so

$$\Delta_t = \arg \min_{\|\Delta\|=1} \{ \langle \mathbf{m}_t, \Delta \rangle \} = \text{LMO}(\mathbf{m}_t),$$

and

$$\begin{aligned} r_t &= \arg \min_{r \geq 0} \left\{ \max \left(\tilde{F}_t + r \langle \mathbf{m}_t, \Delta_t \rangle, F_* \right) + \frac{r^2}{2\eta} \right\} \\ &= \arg \min_{r \geq 0} \left\{ \max \left(\tilde{F}_t - r \|\mathbf{m}_t\|_*, F_* \right) + \frac{r^2}{2\eta} \right\}. \end{aligned}$$

Denote $f(r) = \max \left(\tilde{F}_t - r \|\mathbf{m}_t\|_*, F_* \right) + \frac{r^2}{2\eta}$. Then f can be written piecewise as

$$f(r) = \begin{cases} \tilde{F}_t - r \|\mathbf{m}_t\|_* + \frac{r^2}{2\eta} & r \leq \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \\ F_* + \frac{r^2}{2\eta} & r \geq \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \end{cases}.$$

Note that f is increasing for $r \geq (\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_*$, so its minimizer is the minimizer of $\tilde{F}_t - r \|\mathbf{m}_t\|_* + \frac{r^2}{2\eta}$ for $r \leq (\tilde{F}_t - F_*) / \|\mathbf{m}_t\|_*$. So

$$r_t = \min \left(\eta \|\mathbf{m}_t\|_*, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right),$$

therefore

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \left(\eta, \frac{\tilde{F}_t - F_*}{\|\mathbf{m}_t\|_*} \right) \|\mathbf{m}_t\|_* \text{LMO}(\mathbf{m}_t).$$

Algorithm 2 Momo (Constrained or Regularized)

Inputs: w_0 , learning rate η , momentum β , loss lower bound F_*

```

1: for  $t = 0, 1, \dots, T - 1$  do
2:    $g_t \leftarrow \text{backward}(w_t)$ 
3:    $m_t = \beta m_{t-1} + (1 - \beta)g_t$ 
4:    $\tilde{f}_t = \beta \tilde{f}_{t-1} + (1 - \beta)(F_t(w_t) - \langle g_t, w_t \rangle)$ 
5:    $\tilde{F}_t = \tilde{f}_t + \langle m_t, w_t \rangle$ 
6:   if Constrained then
7:      $w_{t+1} = w_t + \min\left(\eta, \frac{\tilde{F}_t - F_*}{\|m_t\|_*}\right) \text{LMO}(m_t)$ 
8:   else
9:      $w_{t+1} = w_t + \min\left(\eta, \frac{\tilde{F}_t - F_*}{\|m_t\|_*^2}\right) \|m_t\|_* \text{LMO}(m_t)$ 
10:  end if
11: end for

```

Note that this value of w_{t+1} is the unique minimizer of

$$\max\left(\tilde{F}_t + \langle m_t, w - w_t \rangle, F_*\right) + \frac{1}{2\eta}\|w - w_t\|^2,$$

since this function is strongly convex (sum of a convex function and a strongly convex function), and therefore has a unique minimizer. $\square$

The pseudocode for Constrained Momo and Regularized Momo are shown in Algorithm 2. To see why this algorithm correctly computes $\tilde{F}_t$, note that

$$\begin{aligned} \tilde{F}_t &= \sum_{i=0}^t \rho_{t,i} (F_i(w_i) + \langle g_i, w_t - w_i \rangle) \\ &= \sum_{i=0}^t \rho_{t,i} (F_i(w_i) - \langle g_i, w_i \rangle) + \sum_{i=0}^t \rho_{t,i} \langle g_i, w_t \rangle \\ &= \sum_{i=0}^t \rho_{t,i} (F_i(w_i) - \langle g_i, w_i \rangle) + \langle m_t, w_t \rangle. \end{aligned}$$

So denoting $\tilde{f}_t = \sum_{i=0}^t \rho_{t,i} (F_i(w_i) - \langle g_i, w_i \rangle)$, we have $\tilde{F}_t = \tilde{f}_t + \langle m_t, w_t \rangle$, and

$$\tilde{f}_t = \beta \tilde{f}_{t-1} + (1 - \beta)(F_t(w_t) - \langle g_t, w_t \rangle),$$

so that $\tilde{f}_t$ is given by the running average used in Algorithm 2.

Now we derive the closed-form update for our proposed algorithm MuonMax-Momo. Algorithm 3 has the pseudocode for the algorithm, and Proposition 4.3 proves that this procedure implements Regularized Momo with respect to $\|\cdot\|_{\text{MM}}$. Note that Algorithm 3 shows the pseudocode with stale nuclear norm approximations, while Proposition 4.3 considers the vanilla version.

It should be noted that, if we set $\beta = 0$, the stepsize scaling $\sum_{\ell=1}^L \|\mathbf{G}_t^\ell\|_{\text{nuc}}$ for the matrix layers in Algorithm 3 was previously mentioned by Bernstein & Newhouse (2024a) (see their Proposition 5). However, we are not aware of any existing implementation or evaluation of this stepsize scaling, and we found in our experiments that this sort of scaling (without model truncation) is not competitive with Muon.

Algorithm 3 MuonMax-Momo

Inputs: $\mathbf{W}_0^1, \dots, \mathbf{W}_0^L, \boldsymbol{\theta}_0$, learning rates η_m, η_b , EMA parameters β, β_2 , loss lower bound F_*
Defaults: $\eta_m = \eta_b = 0.01, \beta = \beta_2 = 0.95$

 1: **for** $t = 0, 1, \dots, T - 1$ **do**

Update momentum.

 2: $(\mathbf{G}_t^1, \dots, \mathbf{G}_t^L, \mathbf{g}_t^\theta) \leftarrow \text{backward}(\mathbf{W}_t^1, \dots, \mathbf{W}_t^L, \boldsymbol{\theta}_t)$
 3: **for** $\ell = 1, \dots, L$ **do**
 4: $\mathbf{M}_t^\ell = \beta \mathbf{M}_{t-1}^\ell + (1 - \beta) \mathbf{G}_t^\ell$
 5: **end for**
 6: $\mathbf{m}_t^\theta = \beta \mathbf{m}_{t-1}^\theta + (1 - \beta) \mathbf{g}_t^\theta$
 7: $\mathbf{v}_t^\theta = \beta_2 \mathbf{v}_{t-1}^\theta + (1 - \beta_2) \mathbf{g}_t^\theta \odot \mathbf{g}_t^\theta$

Update internal truncation variables.

 8: $\tilde{f}_t = \beta \tilde{f}_{t-1} + (1 - \beta) \left(F_t(\mathbf{W}_t) - \sum_{\ell=1}^L \langle \mathbf{G}_t^\ell, \mathbf{W}_t^\ell \rangle - \langle \mathbf{g}_t^\theta, \boldsymbol{\theta}_t \rangle \right)$
 9: $\tilde{F}_t = \tilde{f}_t + \sum_{\ell=1}^L \langle \mathbf{M}_t^\ell, \mathbf{W}_t^\ell \rangle + \langle \mathbf{m}_t^\theta, \boldsymbol{\theta}_t \rangle$
 10: $d_t = \sqrt{\left(\sum_{\ell=1}^L d_{t-1}^\ell \right)^2 + \frac{\eta_b}{\eta_m} \left\| \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}} \right\|_2^2}$

Update parameters.

 11: **for** $\ell = 1, \dots, L$ **do**
 12: $\mathbf{P} \leftarrow \text{polar}(\mathbf{M}_t^\ell)$
 13: $\mathbf{W}_{t+1}^\ell \leftarrow \mathbf{W}_t^\ell - \min \left(\eta_m, \frac{\tilde{F}_t - F_*}{d_t^2} \right) \left(\sum_{j=1}^L d_{t-1}^j \right) \mathbf{P}$
 14: $d_t^\ell \leftarrow \langle \mathbf{P}, \mathbf{M}_t^\ell \rangle$
 15: **end for**
 16: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \min \left(\eta_b, \frac{\eta_b}{\eta_m} \frac{\tilde{F}_t - F_*}{d_t^2} \right) \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}}$

 17: **end for**

Proposition 4.3. [MuonMax-Momo] Regularized Momo with respect to the norm $\|\cdot\|_{\text{MM}}$ as defined in equation 9 has the following closed form:

$$\begin{aligned} d_t &= \sqrt{\left(\sum_{\ell=1}^L \|\mathbf{M}_t^\ell\|_{\text{nuc}} \right)^2 + \frac{\eta_b}{\eta_m} \left\| \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}} \right\|_2^2} \\ \eta'_m &= \min \left\{ \eta_m, \frac{\tilde{F}_t - F_*}{d_t^2} \right\}, \eta'_b = \min \left\{ \eta_b, \frac{\eta_b}{\eta_m} \frac{\tilde{F}_t - F_*}{d_t^2} \right\} \\ \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t^\ell - \eta'_m \left(\sum_{\ell=1}^L \|\mathbf{M}_t^\ell\|_{\text{nuc}} \right) \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta'_b \frac{\mathbf{m}_t^\theta}{\sqrt{\mathbf{v}_t^\theta + \epsilon}}. \end{aligned}$$

Proof of Proposition 4.3. The proof structure is largely similar to that of Proposition 3.6. By Proposition 4.2, one step of Regularized Momo w.r.t. $\|\cdot\|_{\text{MM}}$ can be written as

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \min \left(\eta_m, \frac{\tilde{F}_t - F_*}{\|\mathbf{M}_t\|_{\text{MM},*}^2} \right) \|\mathbf{M}_t\|_{\text{MM},*} \text{LMO}_{\text{MM}}(\mathbf{M}_t), \quad (21)$$

where $\mathbf{M}_t$ is the momentum buffer for all network parameters, i.e. it is the concatenation of the momentum buffers of each parameter:

$$\mathbf{M}_t = (\mathbf{M}_t^1, \dots, \mathbf{M}_t^L, \mathbf{m}_t^\theta).$$

Comparing Equation 21 with the Proposition statement, we have to prove that $d_t = \|\mathbf{M}_t\|_{\text{MM},*}$ and compute $\|\mathbf{M}_t\|_{\text{MM},*} \text{LMO}_{\text{MM}}(\mathbf{M}_t)$. To do this, we write $\|\cdot\|_{\text{MM}}$ with repeated compositions of norms whose dual and LMO

we already know. Denoting $\lambda = \eta_b/\eta_m$ and

$$\begin{aligned} f(z_1, z_2) &= \sqrt{z_1^2 + \frac{1}{\lambda} z_2^2} \\ g_1(\mathbf{W}_1, \dots, \mathbf{W}_L) &= \max_{\ell \in [L]} \|\mathbf{W}_\ell\|_{2 \rightarrow 2} \\ g_2(\boldsymbol{\theta}) &= \|\boldsymbol{\theta}\|_{\text{ada2}}, \end{aligned}$$

we can write $\|\cdot\|_{\text{MM}}$ as a composition in the notation of Lemma 3.3 as

$$\|\mathbf{W}\|_{\text{MM}} = f(g_1(\mathbf{W}_1, \dots, \mathbf{W}_L), g_2(\boldsymbol{\theta})).$$

Further denoting $\mathbf{D} = \text{diag}(1, 1/\sqrt{\lambda})$, we can write $f(z_1, z_2) = \|\mathbf{D}(z_1, z_2)^T\|_2$. We can now use Lemma 3.3 to compute the dual of $\|\cdot\|_{\text{MM},*}$ as

$$\begin{aligned} \|\mathbf{W}\|_{\text{MM},*} &= f_*(g_{1,*}(\mathbf{W}_1, \dots, \mathbf{W}_L), g_{2,*}(\boldsymbol{\theta})) \\ &\stackrel{(i)}{=} \sqrt{g_{1,*}^2(\mathbf{W}_1, \dots, \mathbf{W}_L) + \lambda g_{2,*}^2(\boldsymbol{\theta})} \\ &\stackrel{(ii)}{=} \sqrt{g_{1,*}^2(\mathbf{W}_1, \dots, \mathbf{W}_L) + \lambda \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2^2} \\ &\stackrel{(iii)}{=} \sqrt{\left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}} \right)^2 + \lambda \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2^2} \end{aligned}$$

where (i) uses Lemma A.1 to plug in the dual of f , (ii) plugs in the dual of $\|\cdot\|_{\text{ada2}}$ which we computed in the proof of Proposition 3.5, and (iii) uses Lemma 3.3 to compute the dual of g_1 , which itself is a composition $g_1 = \ell_\infty \circ (\|\cdot\|_{2 \rightarrow 2}, \dots, \|\cdot\|_{2 \rightarrow 2})$. This confirms that $d_t = \|\mathbf{M}_t\|_{\text{MM},*}$, so

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \min\left(\eta_m, \frac{\tilde{F}_t - F_*}{d_t^2}\right) d_t \text{LMO}_{\text{MM}}(\mathbf{M}_t), \quad (22)$$

To compute the LMO of $\|\cdot\|_{\text{MM}}$, we again use Lemma 3.3. Denoting $(\phi_1, \phi_2) = -\text{LMO}_f(g_{1,*}(\mathbf{W}_1, \dots, \mathbf{W}_L), g_{2,*}(\boldsymbol{\theta}))$, Lemma 3.3 implies

$$\begin{aligned} \text{LMO}_{\text{MM}}(\mathbf{W}) &= (\phi_1 \text{LMO}_{g_1}(\mathbf{W}_1, \dots, \mathbf{W}_L), \phi_2 \text{LMO}_{g_2}(\boldsymbol{\theta})) \\ &\stackrel{(i)}{=} (-\phi_1(\text{polar}(\mathbf{W}_1), \dots, \text{polar}(\mathbf{W}_L)), \phi_2 \text{LMO}_{g_2}(\boldsymbol{\theta})) \\ &\stackrel{(ii)}{=} -\left(\phi_1(\text{polar}(\mathbf{W}_1), \dots, \text{polar}(\mathbf{W}_L)), \phi_2 \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \middle/ \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2 \right), \end{aligned} \quad (23)$$

where (i) uses Lemma 3.3 to compute the LMO of g_1 , which again is the composition $g_1 = \ell_\infty \circ (\|\cdot\|_{2 \rightarrow 2}, \dots, \|\cdot\|_{2 \rightarrow 2})$, and (iii) uses Lemma A.1 to plug in the dual norm of $g_2 = \|\cdot\|_{\text{ada2}}$. The ϕ terms can be simplified as

$$\begin{aligned} (\phi_1, \phi_2) &= -\text{LMO}_f(g_{1,*}(\mathbf{W}_1, \dots, \mathbf{W}_L), g_{2,*}(\boldsymbol{\theta})) \\ &\stackrel{(i)}{=} -\text{LMO}_f\left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}}, \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2\right) \\ &\stackrel{(ii)}{=} -\mathbf{D}^{-1} \text{LMO}_2\left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}}, \sqrt{\lambda} \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2\right) \\ &= \frac{1}{d_t} \mathbf{D}^{-1} \left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}}, \sqrt{\lambda} \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2 \right) \\ &= \frac{1}{d_t} \left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}}, \lambda \left\| \frac{\boldsymbol{\theta}}{\sqrt{\sqrt{\mathbf{v}_t} + \epsilon}} \right\|_2 \right), \end{aligned}$$

where (i) plugs in the previously computed duals $g_{1,*}$ and $g_{2,*}$, and (ii) uses Lemma A.1 to plug in the LMO of f . Plugging the values of (ϕ_1, ϕ_2) into Equation 23 yields

$$\text{LMO}_{\text{MM}}(\mathbf{W}) = -\frac{1}{d_t} \left(\left(\sum_{\ell=1}^L \|\mathbf{W}_\ell\|_{\text{nuc}} \right) (\text{polar}(\mathbf{W}_1), \dots, \text{polar}(\mathbf{W}_L)), \lambda \frac{\boldsymbol{\theta}}{\sqrt{v_t + \epsilon}} \right),$$

and finally, plugging this back into Equation 22 yields

$$\begin{aligned} \mathbf{W}_{t+1}^\ell &= \mathbf{W}_t - \min \left(\eta_m, \frac{\tilde{F}_t - F_*}{d_t^2} \right) \left(\sum_{i=1}^L \|\mathbf{W}_i\|_{\text{nuc}} \right) \text{polar}(\mathbf{M}_t^\ell) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \min \left(\eta_m, \frac{\tilde{F}_t - F_*}{d_t^2} \right) \lambda \frac{\mathbf{m}_t^\theta}{\sqrt{v_t^\theta + \epsilon}} = \boldsymbol{\theta}_t - \min \left(\eta_b, \frac{\eta_b}{\eta_m} \frac{\tilde{F}_t - F_*}{d_t^2} \right) \frac{\mathbf{m}_t^\theta}{\sqrt{v_t^\theta + \epsilon}}, \end{aligned}$$

which is exactly the update in the proposition statement. $\square$

C. Experimental Details for Language Modeling

Setup We did not use weight decay or Nesterov momentum, as we found both to have very small effects on final loss. All methods use a warmup-stable-decay learning rate schedule, where the learning rate is linearly warmed up for the first 5% of steps, held constant until halfway through training, then linearly decayed to 10% of the warmed up value. We use a context length of 1024 and a batch size of 512. Rather than the Newton-Schulz iterations of the original Muon implementation, we use the PolarExpress algorithm (Amsel et al., 2025) to compute approximate polar factors. In this implementation, the weights and gradients are computed in float32, whereas the polar factor is computed in bfloat16 by the PolarExpress (Amsel et al., 2025).

Tuning Protocol For the experiments with FineWeb data in Section 5.1, we tune 36 variations of steepest descent using an iterated grid search to for the two learning rates η_m and η_b . For the 18 variations without model truncation, we first fix the base learning rate at an intermediate value $\eta_b = 1\text{e-}3$, then tune the Muon learning rate with grid search over $\eta_m \in \{1\text{e-}3, 1\text{e-}2, 1\text{e-}1, 1\}$. Some algorithms diverged with $\eta_b = 1\text{e-}3$, and for these algorithms we instead used $\eta_b = 1\text{e-}6$ and searched over $\eta_m \in \{1\text{e-}6, 1\text{e-}5, 1\text{e-}4, 1\text{e-}3\}$. For those algorithms that used $\eta_b = 1\text{e-}6$ for the first phase, we instead search over $\eta_b \in \{1\text{e-}7, 1\text{e-}6, 1\text{e-}5, 1\text{e-}4\}$ in the second phase. Finally, for all of these grid searches, we extend the search space individually for each algorithm until the best LR is not a boundary point of the search space. The resulting tuned LRs are shown in Table 2.

For the 18 variations with model truncation, rather than entirely retuning all algorithms, we reuse the tuned LR ratio η_m/η_b and do a single grid search where η_m and η_b scale together. More specifically, we run each algorithm with LRs $(\rho\eta_m, \rho\eta_b)$, where (η_m, η_b) are the LRs tuned for each algorithm without truncation, and the scaling factor ρ ranges over $\rho \in \{0.3, 1, 3, 10, 30, 100\}$. We found that the best value of ρ for each algorithm was always at least 1 and at most 30. The resulting tuned LRs are shown in Table 3.

Hybrid Norm Definition Recall that Muon-Max is defined as regularized steepest descent with respect to the following norm:

$$\|\mathbf{W}\|_{\text{MM}} = \sqrt{\left(\max_{\ell \in [L]} \|\mathbf{W}^\ell\|_{2 \rightarrow 2} \right)^2 + \frac{\eta_m}{\eta_b} \|\boldsymbol{\theta}\|_{\text{ada2}}^2}.$$

This norm fits into our framework by assigning the spectral norm to each weight matrix $\mathbf{W}_\ell$, assigning $\|\cdot\|_{\text{ada2}}$ to the remaining parameters, and aggregating norms for all parameters with the following "hybrid" outer norm:

$$\|(v_1, \dots, v_L, v_{L+1})\|_{\text{hyb}} = \sqrt{\left(\max_{\ell \in [L]} v_\ell \right)^2 + \frac{\eta_m}{\eta_b} v_{L+1}^2}. \quad (24)$$

D. Additional Language Modeling Results

D.1. FineWeb

The final validation loss reached by all 36 of our evaluated methods is shown in Tables 2 and 3. Each method is denoted as a 3-tuple of settings from our steepest descent framework: regularized vs constrained steepest descent, outer norm, and norm for parameters besides hidden weight matrices.

For the methods without model truncation (Table 2), we see that the RSD methods struggle generally lag behind the CSD methods, likely due to a lack of update normalization. For the CSD methods, Muon and Scion are among the best variations, though the best performing method is actually (Constrained, $\|\cdot\|_{\text{hyb}}, \|\cdot\|_{\text{ada2}}$) (we will return to discuss this method shortly).

For the methods with model truncation (Table 3), we see that both CSD and RSD methods are competitive, meaning that in general model truncation helped RSD methods more than CSD methods (at least in terms of final loss with tuned LR). Muon-Momo has the lowest loss at 3.551 and Scion-Momo is again among the best performers, but actually many methods achieve losses very close to 3.551. Again, we see that (Constrained, $\|\cdot\|_{\text{hyb}}, \|\cdot\|_{\text{ada2}}$) achieves a very low loss, only being outperformed by Muon-Momo.

The method (Constrained, $\|\cdot\|_{\text{hyb}}, \|\cdot\|_{\text{ada2}}$) is quite similar to our proposed method Muon-Max, the only difference being the use of a normalized update. While this method does achieve a lower loss after tuning than MuonMax, we found that this method was not as robust to learning rate tuning. So this method was bested by MuonAdam-Momo in terms of final loss, and it was bested by MuonMax-Momo in terms of learning rate sensitivity, and for this reason we did not perform further evaluations with this method.

Table 2. Final validation losses for all variations without model truncation.

(SD type, Outer Norm, Backup Norm)	Muon LR	Other LR	Final Loss	Name
(Regularized, $\ \cdot\ _{\infty}, \ \cdot\ _{\infty}$)	1e-3	1e-5	3.783	-
(Constrained, $\ \cdot\ _{\infty}, \ \cdot\ _{\infty}$)	1e-2	1e-3	3.599	Scion
(Regularized, $\ \cdot\ _2, \ \cdot\ _{\infty}$)	1e-1	1e-6	4.179	-
(Constrained, $\ \cdot\ _2, \ \cdot\ _{\infty}$)	1e-1	1e-2	3.712	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\infty}$)	1e-3	1e-5	3.826	-
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\infty}$)	1e-2	1e-3	3.610	-
(Regularized, $\ \cdot\ _{\infty}, \ \cdot\ _{\text{ada}\infty}$)	1e-3	1e-5	3.859	-
(Constrained, $\ \cdot\ _{\infty}, \ \cdot\ _{\text{ada}\infty}$)	1e-2	1e-3	3.604	Muon
(Regularized, $\ \cdot\ _2, \ \cdot\ _{\text{ada}\infty}$)	1e-1	1e-4	4.229	-
(Constrained, $\ \cdot\ _2, \ \cdot\ _{\text{ada}\infty}$)	1e-1	1e-2	3.748	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}\infty}$)	1e-3	1e-4	3.917	-
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}\infty}$)	1e-2	1e-2	3.628	-
(Regularized, $\ \cdot\ _{\infty}, \ \cdot\ _{\text{ada2}}$)	1e-3	1e-4	3.757	-
(Constrained, $\ \cdot\ _{\infty}, \ \cdot\ _{\text{ada2}}$)	1e-2	1e-3	3.701	-
(Regularized, $\ \cdot\ _2, \ \cdot\ _{\text{ada2}}$)	1e-1	1e-3	4.049	PolarGrad
(Constrained, $\ \cdot\ _2, \ \cdot\ _{\text{ada2}}$)	1e-1	1e-2	3.664	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada2}}$)	1e-3	1e-3	3.791	MuonMax
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada2}}$)	1e-2	1e-2	3.585	-

Table 3. Final validation losses for all variations with model truncation.

(SD type, Outer Norm, Backup Norm)	Muon LR	Other LR	Final Loss	Name
(Regularized, $\ \cdot\ _\infty, \ \cdot\ _\infty$)	1e-2	1e-4	3.627	-
(Constrained, $\ \cdot\ _\infty, \ \cdot\ _\infty$)	1e-2	1e-3	3.592	Scion-Momo
(Regularized, $\ \cdot\ _2, \ \cdot\ _\infty$)	1	1e-5	3.728	-
(Constrained, $\ \cdot\ _2, \ \cdot\ _\infty$)	1e-1	1e-2	3.843	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _\infty$)	1e-2	1e-4	3.628	-
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _\infty$)	3e-2	3e-3	3.604	-
(Regularized, $\ \cdot\ _\infty, \ \cdot\ _{\text{ada}\infty}$)	3e-2	3e-4	3.578	-
(Constrained, $\ \cdot\ _\infty, \ \cdot\ _{\text{ada}\infty}$)	3e-2	3e-3	3.551	Muon-Momo
(Regularized, $\ \cdot\ _2, \ \cdot\ _{\text{ada}\infty}$)	1	1e-3	3.719	-
(Constrained, $\ \cdot\ _2, \ \cdot\ _{\text{ada}\infty}$)	1e-1	1e-2	3.737	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}\infty}$)	3e-2	3e-3	3.584	-
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}\infty}$)	3e-2	3e-2	3.607	-
(Regularized, $\ \cdot\ _\infty, \ \cdot\ _{\text{ada}2}$)	3e-3	3e-4	3.662	-
(Constrained, $\ \cdot\ _\infty, \ \cdot\ _{\text{ada}2}$)	1e-2	1e-3	3.701	-
(Regularized, $\ \cdot\ _2, \ \cdot\ _{\text{ada}2}$)	3	3e-2	3.613	PolarGrad-Momo
(Constrained, $\ \cdot\ _2, \ \cdot\ _{\text{ada}2}$)	3e-1	3e-2	3.602	-
(Regularized, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}2}$)	1e-2	1e-2	3.576	MuonMax-Momo
(Constrained, $\ \cdot\ _{\text{hyb}}, \ \cdot\ _{\text{ada}2}$)	3e-2	3e-2	3.553	-

Also, we show in Figure 5 results for MuonAdam and MuonMax-Momo on SlimPajama with 6B tokens, an intermediate regime between the 1B and 15B experiments from Section 5. The results are largely similar to those from the 15B runs: MuonMax-Momo exhibits much smaller variation in the loss than MuonAdam as the learning rate changes over multiple orders of magnitude. In fact, MuonMax-Momo outperforms MuonAdam for every learning rate in this range.

We include loss curves for the last 40% of training for the best variations (with tuned learning rates) in Figure 6, and the final losses reached by the best variations (over three seeds) in Table 4. Also, Figure 7 shows a comparison of MuonAdam, Scion, MuonMax against their truncated counterparts.

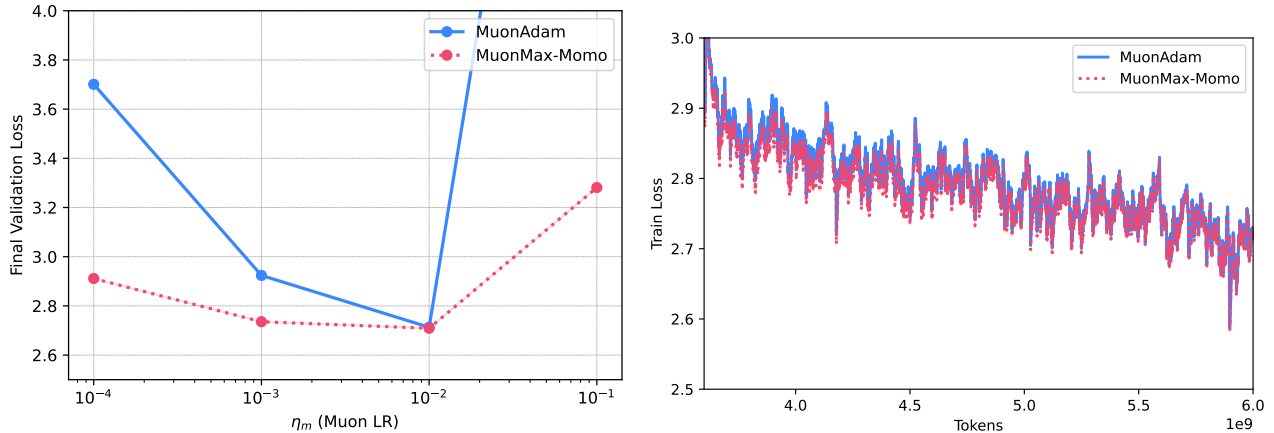


Figure 5. Sensitivity to LR (left) and loss curves for the last 40% of training (right) for SlimPajama6B.

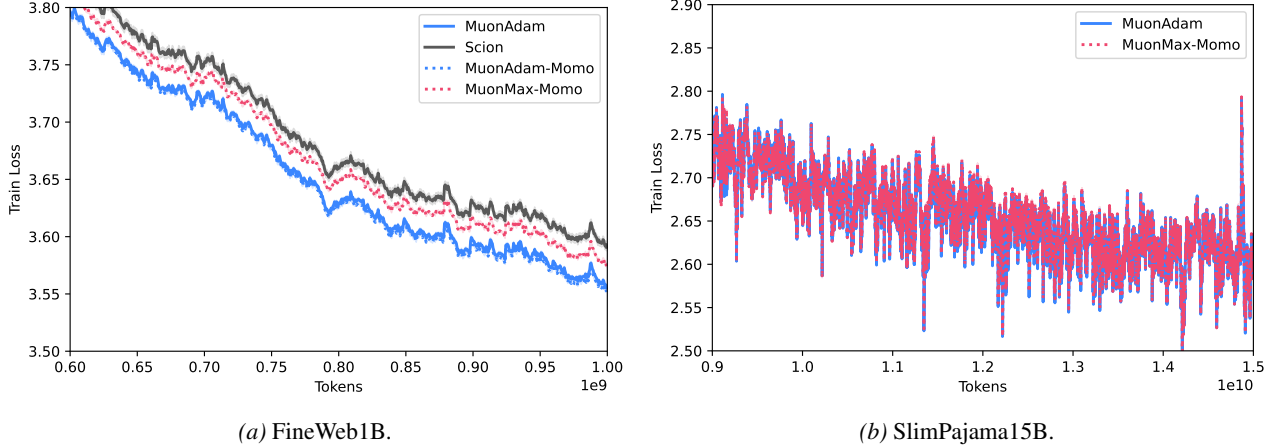


Figure 6. Training loss for the last 40% of training for FineWeb1B (left) and SlimPajama15B (right).

Table 4. Validation loss for FineWeb1B with tuned LRs (mean $\pm$ std over three seeds).

MuonAdam	Scion	MuonAdam-Momo	MuonMax-Momo
3.5592 ± 0.0014	3.5947 ± 0.0031	3.5556 ± 0.0006	3.5779 ± 0.0007

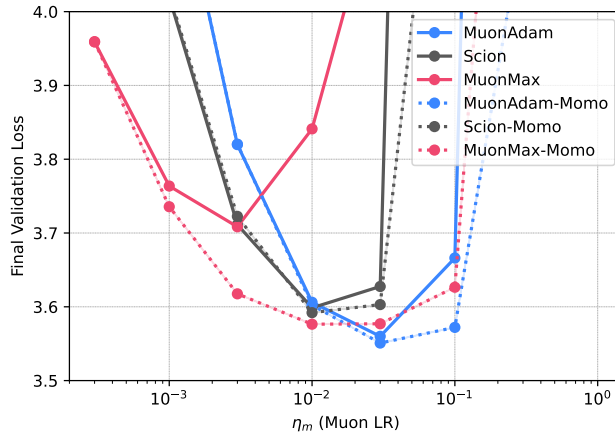


Figure 7. Effect of model truncation on final validation loss. Note that for these runs, we did not use stale nuclear norm approximations in order to isolate the effect of model truncation.

D.1.1. ADDITIONAL BASELINES

Here we evaluate four additional baselines (AdamW, Lion, Adan, Sophia) for the GPT2-Small/FineWeb1B setting. We use the same hyperparameters as outlined in Section C, such as batch size, learning rate schedule, random seeds, etc. For a fair comparison with the Muon-type algorithms from the main paper, we allow the same computational budget for hyperparameter tuning. In particular, since the Muon-type algorithms have two learning rates that were tuned over four possible values each, we tune the learning rate of our additional baselines over 16 values. We also evaluate each additional base both with and without weight decay. For weight decay coefficients and algorithm-specific hyperparameters, we use each baseline’s default settings as reported in their respective papers. The complete search range and tuned values for each algorithm are shown in Table 5.

The losses for each additional baseline are shown in Figure 8. The most important feature to notice is that none of these baselines reach as low a loss as the baselines from the main paper: from Figure 3a, all of MuonAdam, Scion, MuonAdam-

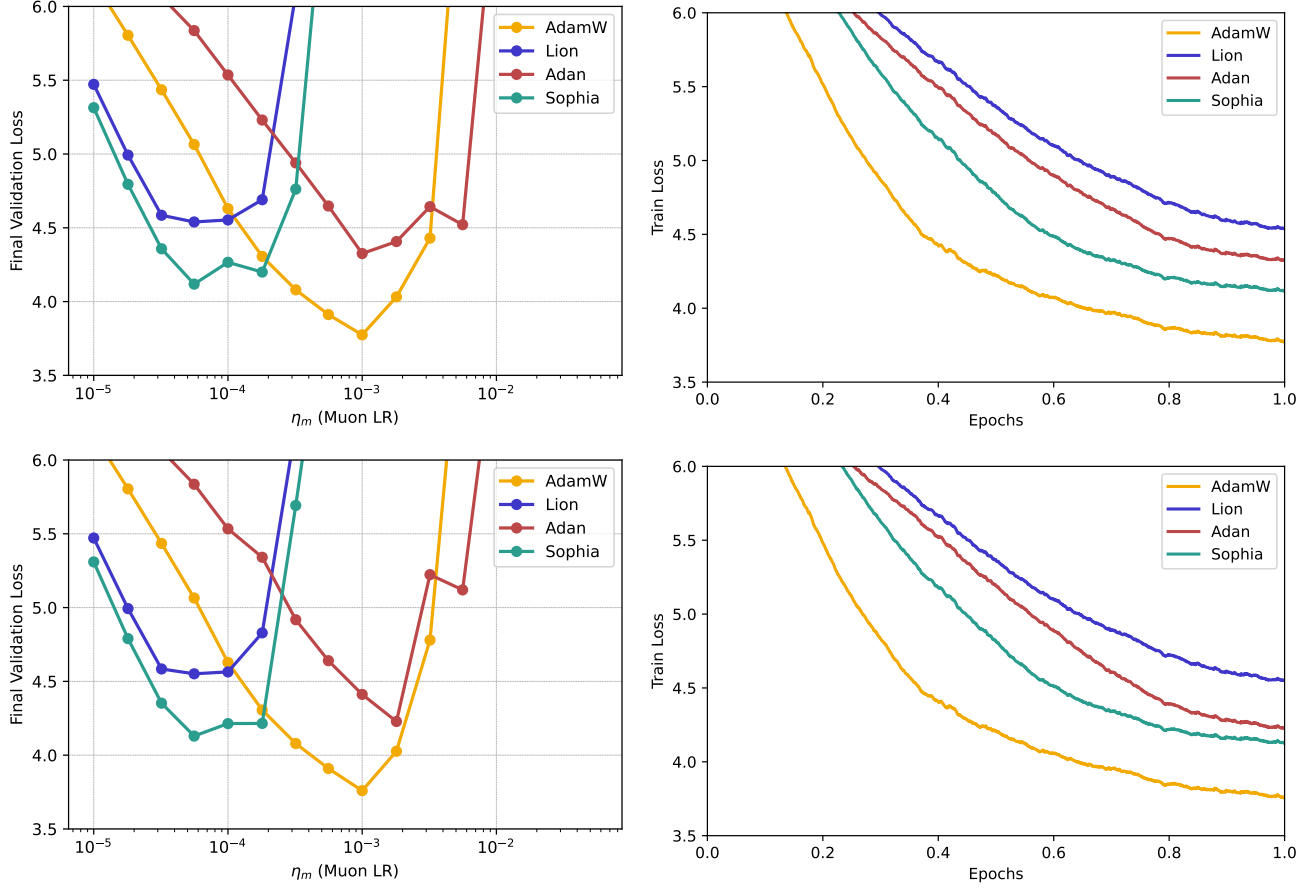


Figure 8. Loss for additional baselines when training GPT2-Small on FineWeb1B. Top row shows results with weight decay, bottom row without weight decay. Left: Validation loss for a sweep over learning rates. Right: Training loss curves for each baseline with tuned learning rate.

Momo, and MuonMax-Momo achieve less than 3.6 loss with a tuned LR and less than 3.75 loss with multiple LR in the grid. All of the additional baselines except for AdamW never get below 4.0 validation loss, and AdamW at its best reaches only 3.759. Note also that each of the four baselines were run with and without weight decay, which did not make a significant difference. This means that each baseline was allowed a total of 32 hyperparameter configurations, which is significantly more than the Muon-type algorithms from Figure 3a. We conclude that AdamW is the only additional baseline that is competitive with the aforementioned Muon-type algorithms, though it is still decisively outperformed.

Table 5. Hyperparameter information for additional baselines. For all baselines, we tune the learning rate over $10^{-5+i/4}$ for $i \in \{0, \dots, 15\}$ once with weight decay on, and again with weight decay off. For each baseline, we set the weight decay coefficient λ according to the default value for that algorithm, listed below.

	Tuned LR ($\lambda = 0$)	Tuned LR ($\lambda > 0$)	Alg-specific parameters
AdamW	$\eta = 1\text{e-}3$	$\eta = 1\text{e-}3$ ($\lambda = 0.01$)	$\beta_1 = 0.95, \beta_2 = 0.95$
Lion	$\eta = 5.6\text{e-}5$	$\eta = 5.6\text{e-}5$ ($\lambda = 0.01$)	$\beta_1 = 0.95, \beta_2 = 0.98$
Adan	$\eta = 1.8\text{e-}3$	$\eta = 1\text{e-}3$ ($\lambda = 0.02$)	$\beta_1 = 0.02, \beta_2 = 0.08, \beta_3 = 0.01$
Sophia	$\eta = 5.6\text{e-}5$	$\eta = 5.6\text{e-}5$ ($\lambda = 0.2$)	$\beta_1 = 0.965, \beta_2 = 0.99, \rho = 0.05,$ Hessian estimator: GNB

Table 6. Additional metrics for GPT2-Small/FineWeb1B, including the additional baselines from Section D.1.1. The target loss for this setting is 3.8, which we chose to be small enough to discriminate between the best methods and large enough to include AdamW.

	Best Perplexity	Throughput (tok/s)	Time per step (s)	Time to target loss (s)	Tokens to target loss	LR Basin Width
MuonAdam	33.21	938K	0.559	599.06	548M	1.523
Scion	34.40	948K	0.554	661.62	575M	1.477
MuonAdam-Momo	33.14	901K	0.582	648.55	548M	2.523
MuonMax-Momo	33.95	892K	0.588	637.00	553M	3.523
AdamW	40.99	990K	0.530	872.78	818M	1.000
Lion	92.11	996K	0.527	1015.37	999M	1.000
Adan	66.46	957K	0.548	1083.54	999M	0.750
Sophia	59.48	974K	0.539	1020.79	999M	1.250

D.1.2. ADDITIONAL METRICS

We further quantify the efficiency and learning rate sensitivity of each method in Table 6. Efficiency is measured in terms of token throughput, time per training step, time and tokens to reach a target loss, and the “width” of the basin of the LR sensitivity curves. We define the LR basin width as follows: for a learning rate sweep of a particular method, let ℓ be the loss achieved by the best learning rate, and let η_0 (respectively η_1) be the smallest (respectively largest) learning rate in the grid that achieves less than 1.2ℓ loss. The LR basin width is then defined as $\log_{10}(\eta_1/\eta_0)$. Essentially, the basin width quantifies the orders of magnitude by which the learning rate can vary while still achieving a reasonable loss of 1.2 times the optimum. For throughput and time per step, we average over all learning rates, while for time-to-target, tokens-to-target, and perplexity we choose the best value over all learning rates. Similar metrics for our GPT2-Large/SlimPajama experiments are given in Tables 8 and 9 in Section D.2.1.

From Tables 6, 8, 9, we see that the best perplexity is always achieved by either MuonAdam-Momo or MuonMax-Momo, and these two algorithms significantly outperform all baselines in terms of learning rate sensitivity as quantified by the LR basin width. Compared to MuonAdam, our two proposed algorithms take about 5% more time per step for GPT2-Small and 1.5% to 3% more for GPT2-Large. Also, with tuned LRs, our two proposed algorithms reach the target loss with as many or fewer tokens than all other baselines. The time to target loss of our algorithms with tuned LRs is sometimes better, sometimes worse than MuonAdam, but the gap is never more than 5% for MuonMax-Momo and 10% for MuonAdam-Momo. These results reinforce our finding that our proposed algorithms are significantly more robust to learning rate tuning than baselines, with similar or improved performance after tuning and only a modest increase in wall-clock time.

D.1.3. VARYING BATCH SIZE

In this section, we perform learning rate sweeps for MuonAdam, Scion, MuonAdam-Momo, and MuonMax-Momo with varying batch sizes. We evaluate these algorithms on the GPT2-Small/FineWeb1B setup detailed in Sections 5.1 and C. Previously we used a batch size of 512; in this section, we vary the batch size over $\{128, 256, 512, 1024, 2048\}$ while keeping all other settings the same. Note that we reuse the tuned LR ratio η_m/η_b as detailed in Section C.

For each batch size, the final validation loss reached by each algorithm with varying learning rate is shown in Figure 9. Overall, we find the results with different batch size to be largely consistent with our previous results. For all batch sizes, both MuonAdam-Momo and MuonMax-Momo have a wider range of competitive learning rates than MuonAdam and Scion, and for all batch sizes greater than 512, both MuonAdam-Momo and MuonMax-Momo outperform the baselines for every learning rate we tried.

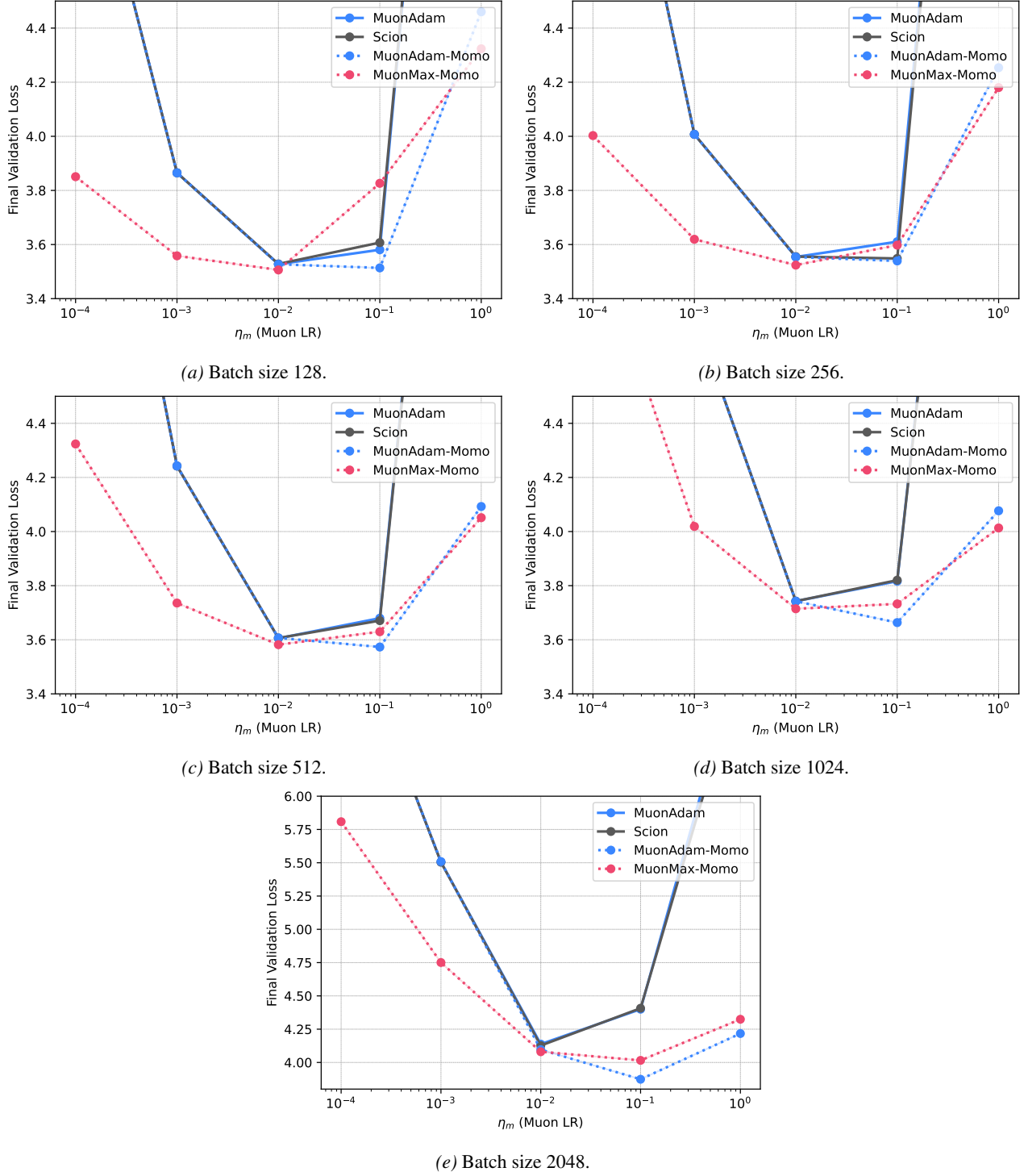


Figure 9. Learning rate sensitivity for GPT2-Small/FineWeb1B with varying batch sizes.

Architecture Component	Qwen2-MoE (Small)	Qwen2-MoE (Medium)
Hidden size	256	768
Decoder layers	12	16
Attention heads	4	12
Head dimension	64	64
Experts per layer	8	4
Routing	Top-2	Top-2
Per-expert FFN width	1280	4096
Shared expert width	1536	4096
Embedding tying	Tied	Tied
Context length	1024	1024
Micro-batch size	32	32
Global batch size	512	512
Total # params	124,582,400	831,407,616

Table 7. Architectural specifications of the two Qwen2 Mixture-of-Experts (MoE) models used in our experiments.

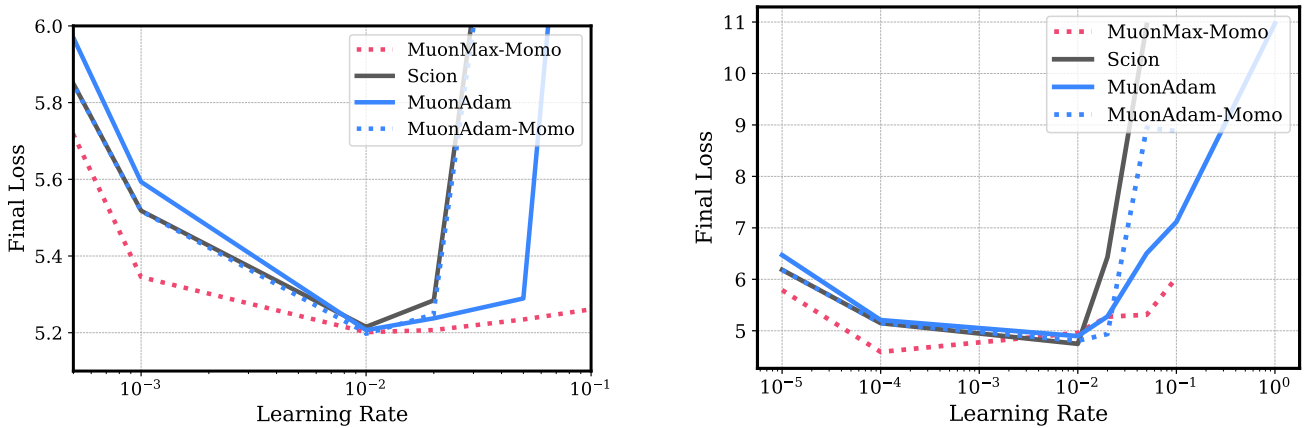


Figure 10. Final loss of Qwen2-MoE for FineWeb1B. Left: Small architecture, 1B tokens. Right: Medium architecture, 10B tokens.

D.1.4. QWEN2MOE MODEL

We also compared the best performing methods MuonAdam, Scion, MuonMax-Momo and MuonAdam-Momo, when training a Mixture of Experts type model. We trained two Qwen2-MoE variants on the FineWeb dataset, one small model with $\mathcal{O}(124 \times 10^6)$ parameters and one medium sized model with $\mathcal{O}(831 \times 10^6)$ parameters, see Table 7 for details. These configurations emphasize MoE capacity while keeping active parameters per token modest for rapid ablation, following prior sparse expert designs (Shazeer et al., 2017; Fedus et al., 2022; Du et al., 2022). Qwen2-MoE (Yang et al., 2024) architectural choices and implementation details are taken from the official Qwen2 repository <https://github.com/QwenLM>. No intermediate validation was performed (only end-of-epoch loss) to minimize overhead.

For the two Momo methods we use $F^* = 3.2$ without tuning or trying other values. This turned out to be far from the loss achieved by either the small or medium MoE model. For all methods, we set the learning rate ratio $\eta_m/\eta_b = 1$.

The final loss for each algorithm across a range of learning rates is shown in Figure 10. Indeed, again we found MuonMax-Momo to be the most stable method; for the small variant, MuonMax-Momo achieves the lowest loss for every learning rate we tried. The best loss over all learning rates achieved by each method is given in the following table.

	Qwen2-MoE Small	Qwen2-MoE Medium
MuonAdam	5.2072	4.8995
Scion	5.2156	4.7504
MuonAdam-Momo	5.1975	4.8184
MuonMax-Momo	5.2017	4.6561

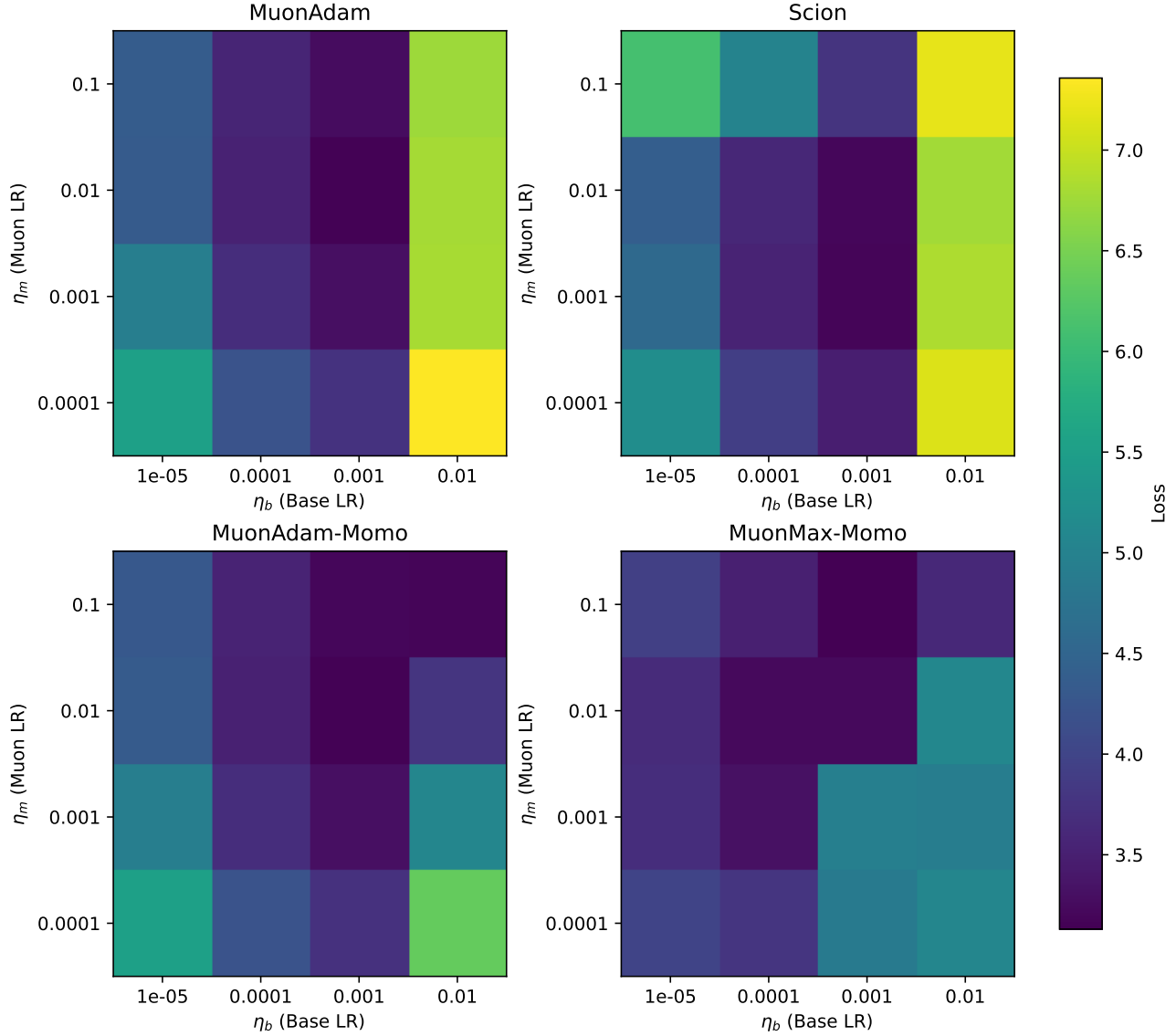


Figure 11. 2D learning rate sensitivity for SlimPajama1B.

D.2. SlimPajama

Figure 11 shows a 2D visualization of final validation losses for Muon, Scion, MuonAdam-Momo, and MuonMax-Momo as the two learning rates vary. We find MuonMax-Momo to be most stable to changes in the learning rates, with both Muon and Scion suffering high losses when the base LR η_b is large. Interestingly, Muon-Momo has the highest loss when the Muon LR η_m is small and the base LR η_b is large.

We also include loss curves for the last 40% of training for MuonAdam and MuonMax-Momo (with tuned learning rates) in Figure ??.

D.2.1. ADDITIONAL METRICS

Similarly to Section D.1.2, here we include additional metrics to quantify efficiency and LR robustness for our SlimPajama experiments. The definition of each metric is the same as Section D.1.2, here we only change the target loss to 3.3 for GPT2-Large/SlimPajama1B and 3.0 for GPT2-Large/SlimPajama6B. The results are shown in Tables 8 and 9, and are largely similar to those of Table 6. See Section D.1.2 for a discussion of Tables 6, 8, and 9 together.

Table 8. Additional metrics for GPT2-Large/SlimPajama1B. The target loss for this setting is 3.3.

	Best Perplexity	Throughput (tok/s)	Time per step (s)	Time to target loss (s)	Tokens to target loss	LR Basin Width
MuonAdam	25.40	126K	4.138	3172.77	389M	1.000
Scion	26.69	127K	4.114	4273.48	530M	1.000
MuonAdam-Momo	25.37	124K	4.218	3250.07	389M	4.000
MuonMax-Momo	24.84	125K	4.178	3146.43	389M	5.000

Table 9. Additional metrics for GPT2-Large/SlimPajama6B. The target loss for this setting is 3.0.

	Best Perplexity	Throughput (tok/s)	Time per step (s)	Time to target loss (s)	Tokens to target loss	LR Basin Width
MuonAdam	17.29	127K	4.124	4319.90	531M	1.000
MuonMax-Momo	17.21	123K	4.239	4308.02	531M	2.000

E. Image Classification Experiments

In order to benchmark our proposed algorithms in a variety of settings, here we evaluate our proposed algorithms and baselines for image classification tasks. We evaluate SGD-M (SGD with momentum), AdamW, MuonAdam, Scion, MuonAdam-Momo, and MuonMax-Momo, first for training a ResNet20 (He et al., 2016) on CIFAR-10, then for a ResNet110 on CIFAR-100.

Setup All methods use a batch size of 128 and cross entropy loss. We train for 50 epochs with ResNet20/CIFAR-10 and 100 epochs with ResNet110/CIFAR-100. We evaluate with two different learning rate schedules (constant and cosine) and we do not use weight decay. For SGD-M we set the momentum to 0.9. For AdamW, MuonAdam, MuonAdam-Momo, and MuonMax-Momo, we set $\beta_1 = \beta_2 = 0.95$. We use standard data augmentation for CIFAR: normalization, random horizontal flipping, and random cropping. For Muon-type algorithms, we assign the spectral norm to weights of convolutional layers except for the first convolution in the network. We interpret each convolutional filter as a matrix by flattening all dimensions after the first two, following an early implementation of Muon for image classification (Jordan, 2024).

For the algorithms with two learning rates, i.e. the Muon-type algorithms, we tune the ratio between learning rates with a double grid search. The base LR η_b is tuned over $\{1e-5, 1e-4, 1e-3, 1e-2\}$, and the Muon LR η_m is tuned over $\{1e-4, 1e-3, 1e-2, 1e-1\}$. To benchmark the learning rate sensitivity with a 1D sweep, we then fix the ratio η_m/η_b and vary η_b over $\{1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1\}$. For SGD-M and AdamW, we sweep the learning rate over $\{1e-3, 1e-2, 1e-1, 1, 10\}$ and $\{1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$, respectively. Note that we tune learning rates separately for the two schedules (constant, cosine). For the final sweep, we run three random seeds for each algorithm/LR.

Results The final training losses and validation accuracies for CIFAR-10 and CIFAR-100 with both constant and cosine LR are shown in Figures 12 through 15. The plots show the mean plus/minus one standard deviation across three seeds.

For CIFAR-10, MuonMax-Momo and MuonAdam-Momo jointly achieve the lowest training loss, with MuonAdam-Momo slightly better with cosine LR. MuonAdam and Scion are competitive in terms of validation accuracy, and their LR basins are widest among all the methods with constant LR, but they don’t reach as low a loss as our proposed methods. Here, MuonMax-Momo has achieved a balance between best loss and learning rate sensitivity: it reaches nearly the same tuned loss as MuonAdam-Momo while enjoying a wider LR basin. In fact, with cosine LR, MuonMax-Momo has the widest LR basin among all methods. SGD-M and AdamW lag behind in terms of both training loss and validation accuracy.

For CIFAR-100, all algorithms appear more sensitive to the choice of learning rate. With constant LR, MuonMax-Momo achieves the lowest loss and highest validation accuracy after tuning, while MuonAdam achieves the lowest loss after tuning with cosine LR. Again, MuonAdam-Momo is more sensitive than MuonMax-Momo with constant LR.

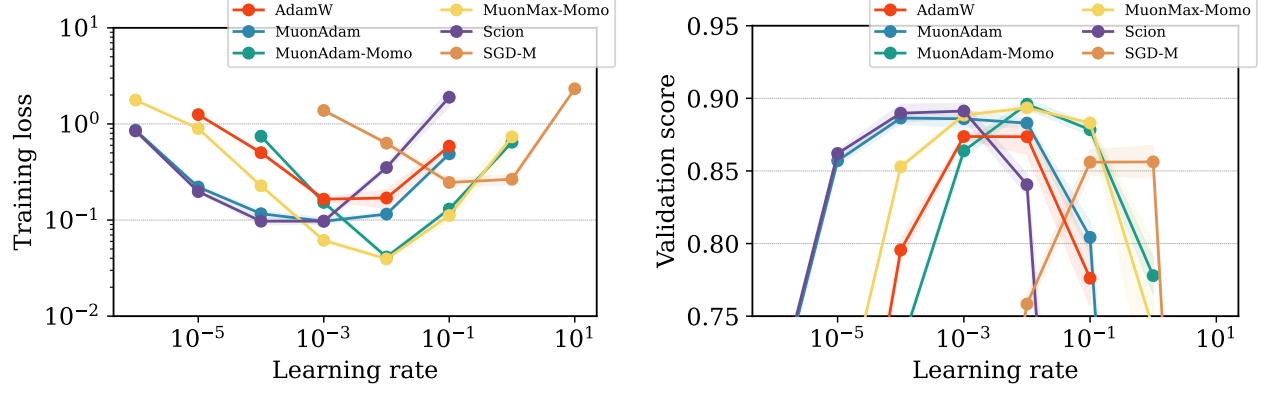


Figure 12. CIFAR-10 with ResNet20, constant learning rate. Note that the x-axis shows the base learning rate η_b , while the Muon learning rate η_m is set according to the tuned ratio η_m/η_b .

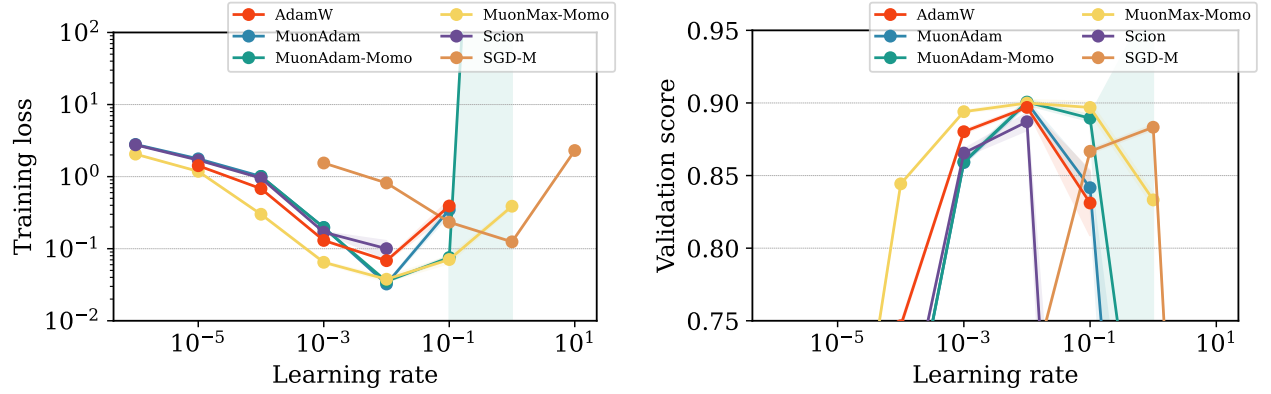


Figure 13. CIFAR-10 with ResNet20, cosine learning rate schedule. Note that the x-axis shows the base learning rate η_b , while the Muon learning rate η_m is set according to the tuned ratio η_m/η_b .

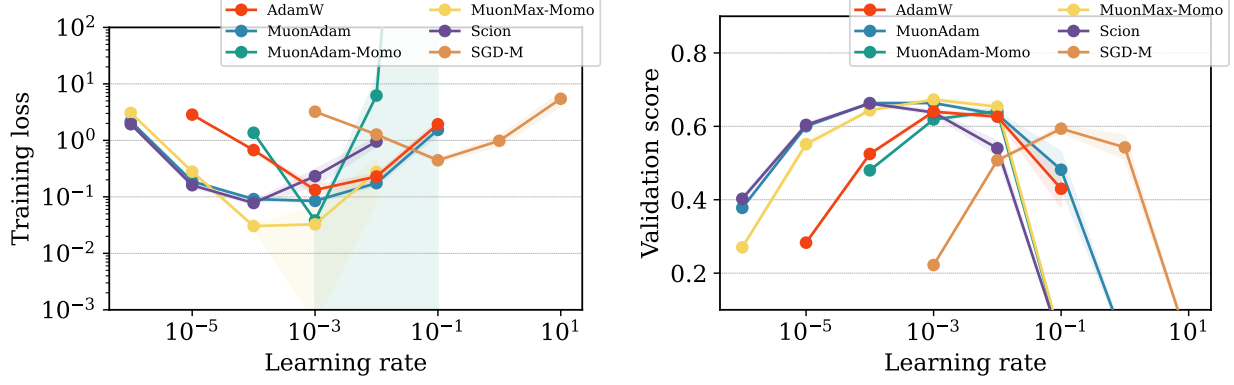


Figure 14. CIFAR-100 with ResNet110, constant learning rate. Note that the x-axis shows the base learning rate η_b , while the Muon learning rate η_m is set according to the tuned ratio η_m/η_b .

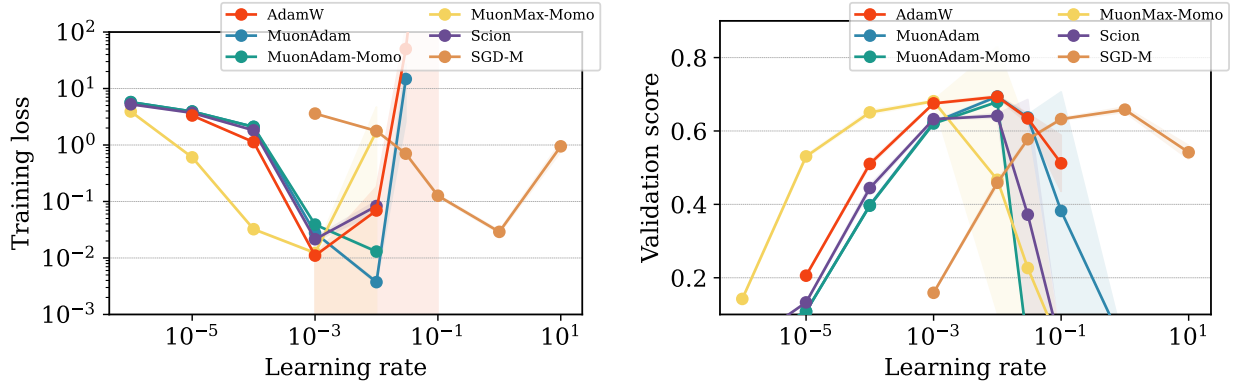


Figure 15. CIFAR-100 with ResNet110, cosine learning rate schedule. Note that the x-axis shows the base learning rate η_b , while the Muon learning rate η_m is set according to the tuned ratio η_m/η_b .